

LABELGUARD

SIH26034 — 5-Day AI-Assisted Software Build Handbook

“Scan the Label. Find the Violation. Show the Evidence.”

A Complete Zero-to-Prototype Playbook for Six Hardware Students Transitioning to Software

VERSION	1.0
DOCUMENT DATE	28 August 2026
SOURCE VERIFICATION DATE	28 August 2026 (live web search)
TRACK / THEME	Software — Agriculture, FoodTech & Rural Development

Table of Contents

How to Use This Handbook.....	4
Part 1 — Executive Summary.....	5
Part 2 — Problem Statement Deconstruction.....	9
Part 3 — User Personas.....	11
Part 4 — The Exact MVP.....	13
Part 5 — Product Workflow.....	15
Part 6 — Product UI.....	19
Part 7 — The Signature Feature: Evidence-First Compliance.....	22
Part 8 — Human-in-the-Loop System.....	23
Part 9 — Legal Source of Truth.....	24
Part 10 — Rule Engine.....	27
Part 11 — OCR / Computer Vision.....	29
Part 12 — AI / LLM Architecture (Where AI Belongs).....	31
Part 13 — Exact Tech Stack.....	32
Part 14 — Official Learning Resources.....	35
Part 15 — LLM Tool Strategy.....	36
Part 16 — The Golden LLM Prompting System.....	38
Part 17 — LLM Debugging Playbook.....	42
Part 18 — Member-by-Member Handbook.....	45
Part 19 — Five-Day Master Schedule.....	57
Part 20 — Daily War Room.....	58
Part 21 — GitHub Workflow.....	59
Part 22 — Testing Strategy.....	61
Part 23 — Dataset Strategy.....	63
Part 24 — Performance Metrics.....	64
Part 25 — Threat / Failure / Loophole Analysis.....	65
Part 26 — Security.....	71
Part 27 — Privacy / Ethics.....	72
Part 28 — Business Model.....	73
Part 29 — Competitor Analysis.....	75
Part 30 — Competitive Moat.....	77
Part 31 — Pitch Deck (10 Slides).....	78
Part 32 — 3-Minute Pitch (Word-for-Word).....	81

Part 33 — 90-Second Demo Sequence.....	82
Part 34 — Judge Q&A (50+ Questions).....	83
Part 35 — Research Paper Outline.....	87
Part 36 — Limitations.....	88
Part 37 — Future Roadmap.....	89
Part 38 — Disaster Recovery.....	90
Part 39 — “If You Are Stuck” Decision Tree.....	91
Part 40 — Copy-Paste Prompt Library (100+ Prompts).....	93
Part 41 — LLM Context Files.....	100
Part 42 — Claude / Coding Agent Workflow.....	102
Part 43 — Code Review Checklist.....	103
Part 44 — Beginner Glossary.....	104
Part 45 — Exact Command Cheatsheet.....	106
Part 46 — Daily Checklist (Printable).....	107
Part 47 — Definition of Done.....	111
Part 48 — Feature Priority Matrix (MoSCoW).....	112
Part 49 — Risk Register.....	113
Part 50 — SIH Judge Psychology.....	116
Part 51 — What Will Make This Project Lose.....	117
Part 52 — What Will Make This Project Stand Out.....	118
Part 53 — Final Project Blueprint.....	119
Part 54 — Final Five-Day War Plan.....	120
Part 55 — Final Emergency Mode.....	121
Part 56 — References.....	122
Part 57 — Resource Priority.....	123
Part 58 — Appendices.....	124
Final Quality Self-Audit.....	126

How to Use This Handbook

This is a lookup manual, not a novel. You are not expected to read it cover to cover before starting. The intended pattern is:

Find your role in **Part 18** → find today's date in your role's 5-day table → do the Morning/Afternoon/Evening tasks → if the detailed breakout box exists for that task, follow its Objective → Prompt → Test → Definition of Done sequence exactly.

Stuck on something specific? Go straight to **Part 17** (LLM Debugging Playbook) or **Part 39** (Decision Trees) and find your exact symptom — don't read the whole section.

Building the pitch? Parts **31–34** are self-contained — deck, 3-minute script, 90-second demo, and Q&A prep.

Worried about legal accuracy? **Part 9** is the single source of truth every rule must trace back to. Read it before writing a single rule in **Part 10**.

NOTE — The one architectural fact to never forget

An LLM never makes the final compliance decision in this system. A deterministic, source-cited rule engine does. Every part of this handbook is written to protect that boundary — see Part 12.

Source-category shorthand used throughout (see Part 2 for the full legend): [A] official/legal source · [B] technical implementation decision · [C] prototype assumption · [D] future production requirement · [E] inference.

Part 1 — Executive Summary

1.1 What SIH26034 Actually Asks For

Problem statement ID: SIH26034

Sponsor: Ministry of Consumer Affairs, Food & Public Distribution — Department of Consumer Affairs

Ask: “Software System to check compliance of Packaged Commodities under Legal Metrology (Packaged Commodities) Rules, 2011 by scanning products, images and labels.”

In plain language: the government wants a software tool that looks at a photo of a packaged product's label and tells an inspector whether the required information (manufacturer name, net quantity, price, date of manufacture, consumer-care details, country of origin, etc.) is present, correctly formatted, and legible — flagging anything that looks wrong so a human inspector can verify it faster than doing it by hand.

1.2 Why It Matters

- India has millions of SKUs (stock-keeping units) on shelves and e-commerce listings. Legal Metrology field officers currently check labels manually, one package at a time.
- Manual checks are slow, inconsistent between inspectors, and easy to get wrong on small details (font size, missing declarations, incomplete addresses).
- A scanning tool that pre-screens packages lets a human inspector cover far more ground per day and builds a documented, photographic evidence trail for every flagged case.

1.3 Who the Users Are

Primary user: a Legal Metrology field inspector or an inspection supervisor reviewing flagged cases. Secondary users: administrative officers compiling inspection statistics, and (as a future extension only) manufacturers who want to self-check a label before printing.

1.4 The Problem That Exists Today

Inspection is manual, paper-heavy, and inconsistent. There is no standard, photographic, field-level tool that converts a package photo into a structured compliance checklist with evidence. Existing commercial “AI label verification” tools (Part 29) are built for brand-owner pre-print artwork QC in FMCG factories — not for a government field inspector checking a product that is already on a shelf, against India's specific Legal Metrology declarations.

1.5 What LabelGuard Does

1. Inspector uploads or photographs a packaged product.
2. System checks image quality and asks for a retake if it's unusable.
3. OCR reads all text on the visible label.
4. The read text is parsed into structured fields (manufacturer name, net quantity, MRP, date of manufacture, consumer care, country of origin, etc.).
5. Each field is normalized (units, currency symbols, date formats).

6. An explicit, human-written rule engine — not an LLM — checks each field against the applicable Legal Metrology declaration requirement.
7. Every flagged issue is linked back to the exact region of the image that triggered it, with a confidence score.
8. Low-confidence results are routed to manual review instead of being auto-decided.
9. The inspection is saved to history and a PDF report is generated.

1.6 What Makes LabelGuard Different

- Evidence-first: every violation shows the exact image region and field it came from — never a bare “non-compliant” verdict.
- Deterministic legal decisions: compliance is decided by an explicit, versioned rule engine referencing the actual Rules, not by an LLM guessing.
- Confidence-aware: the system openly separates what it's sure of from what needs a human to look.
- Built for the Indian regulation specifically (Legal Metrology (Packaged Commodities) Rules, 2011), not a generic Western FMCG QC tool repurposed for India.

1.7 What the MVP Will and Will Not Do

The 5-day MVP WILL	The 5-day MVP will NOT (yet)
Check a fixed, documented set of machine-checkable declarations (Part 9)	Attempt to verify declarations that legally require judgment calls only a trained officer can make
Show evidence + confidence for every result	Auto-issue a legal notice or penalty
Work on a small, self-built demo image set	Claim government-grade validated accuracy
Run on one laptop/one server, single language, single frontend + backend	Support real-time video, multi-store rollout, or offline mobile from day one

1.8 Why This Is Buildable in Five Days

Every subsystem (upload → OCR → rule check → evidence → report) is a well-documented, off-the-shelf pattern with mature open-source libraries (Part 13). Nothing needs to be invented from scratch. The team is not training a model — it is wiring together an OCR library, a small rules file, and a CRUD web app. That is realistic for six people in five days with heavy LLM coding assistance, provided scope stays inside the MVP boundary in Part 4.

1.9 Why a Hardware Team Fits This Project

- Hardware engineers are already comfortable with structured, deterministic systems (rules, thresholds, signal pipelines) — the rule engine and OCR pipeline are conceptually similar to a sensor-processing pipeline.
- The project explicitly avoids requiring deep prior software/web experience — LLMs handle syntax; the team supplies structure, testing discipline, and rule logic.
- No physical hardware, PCB, or firmware risk exists in the critical path, which removes the single biggest reason hardware prototypes fail to demo on time.

1.10 What the Final Demo Should Look Like

Live, three-package demo (Part 33): one compliant package sails through green; one non-compliant package is flagged with a highlighted image region, a plain-language explanation, and a confidence score; one ambiguous/low-quality package is correctly routed to “manual review required” instead of a false verdict. The judge sees the PDF report generated at the end. Total demo time: 90 seconds, backed by a pre-recorded fallback video in case Wi-Fi fails (Part 38).

1.11 Explaining LabelGuard at Different Lengths

30-Second Explanation

“LabelGuard scans a photo of a packaged product and checks it against India's Legal Metrology labeling rules — manufacturer details, net quantity, price, date, origin. It shows exactly which rule was broken and highlights the evidence on the image itself, so a human inspector can confirm it in seconds instead of checking every line by hand.”

60-Second Explanation

“Under India's Legal Metrology (Packaged Commodities) Rules, 2011, every packaged product must declare specific information — who made it, how much is inside, what it costs, when it was made, and how to reach consumer care. Today, checking this is manual: an inspector reads each label by eye. LabelGuard automates the first pass. An inspector photographs a product; the system runs OCR to read the label, extracts the required fields, and checks them against an explicit rule engine built from the actual Rules — not a guess from an AI model. If something looks wrong, LabelGuard shows exactly where on the image and how confident it is. Anything uncertain is sent for manual review, never auto-decided. The result is a structured inspection report the officer can file immediately, dramatically increasing how many products a single inspector can screen per day.”

3-Minute Explanation

Walk through: the regulatory problem → today's manual workflow → the LabelGuard pipeline end-to-end (upload → quality check → OCR → extraction → rule engine → evidence → human review → report → history) → why an LLM is never the final legal decision-maker → the three-package live demo → the honest limitations (OCR on curved/glare/tiny text, legal ambiguity) → the roadmap from prototype to a real departmental pilot. Use this as the spoken skeleton for the full pitch in Part 32.

Technical Explanation

A FastAPI backend receives an uploaded image, runs a blur/glare/resolution quality gate, passes the image to an OCR engine (PaddleOCR, chosen over Tesseract for this use case — see Part 11) which returns text plus bounding boxes and per-token confidence. A field-extraction layer (regex + light heuristics, optionally LLM-assisted for normalization only) maps raw OCR tokens to a structured JSON object of declared fields. This JSON is evaluated against a versioned, human-authored JSON/Python rule set (Part 10) that encodes machine-checkable requirements from the Rules. Each rule execution returns a pass/fail/uncertain verdict, a severity, a confidence score, and the source bounding box. Results below a confidence threshold are routed to a manual-review queue. A Next.js frontend renders the evidence-first UI. Inspection records persist in SQLite. A PDF report is generated with ReportLab.

Non-Technical Explanation

Think of LabelGuard as a very fast, very literal assistant that reads the fine print on a package for the inspector, compares it against a checklist drawn from the actual law, circles anything on the photo that looks off, and tells the inspector honestly when it isn't sure — so a human always makes the final call, but has to do far less manual reading to get there.

Part 2 — Problem Statement Deconstruction

Deconstructing SIH26034 phrase by phrase, using only what the statement and the official Rules actually say (Category A/B below is used throughout this handbook — see legend).

NOTE — Source-Category Legend (used throughout this handbook)

A = Official/legal source (the Rules, gazette notification, or Department of Consumer Affairs page).

B = Technical implementation decision made by this team for the prototype.

C = Prototype assumption (true for the demo, not guaranteed in general).

D = Future production requirement, out of scope for the 5-day build.

E = Inference/interpretation — reasonable but not stated verbatim in a source.

Phrase-by-phrase reading

Phrase in PS	What it means (source category noted)
“Software System”	A standalone application (web app, in this build), not a hardware device. [B]
“check compliance”	Compare declared/observed label content against a rule set derived from the Rules and report pass/fail/uncertain — not issue a legal certification. [A+B]
“Packaged Commodities”	Pre-packed goods intended for retail sale, as defined and scoped by the Legal Metrology Act, 2009 and the 2011 Rules made under it. [A]
“Legal Metrology”	The Indian regulatory regime (Legal Metrology Act, 2009) governing weights, measures, and packaged-commodity declarations, administered by the Department of Consumer Affairs. [A]
“Rules, 2011”	The Legal Metrology (Packaged Commodities) Rules, 2011 (in force from 1 April 2011), as amended since — the primary source of truth for this project (Part 9). [A]
“scanning products, images and labels”	Input is visual: a photograph of the product/label, not a barcode feed or manual data entry. Implies OCR + computer vision as the core technical pipeline. [B/E]

Requirement → Product Feature Mapping

Requirement (from PS/Rules)	Product Feature
Scanning products	Image upload + (should-have) camera capture
Scanning labels	Label region focus + OCR
Compliance	Deterministic rule engine (Part 10)
Inspection	Inspection record with timestamp, image, officer note
Evidence	Bounding-box-highlighted image region per finding

Requirement (from PS/Rules)	Product Feature
Reporting	Structured PDF inspection report
Human oversight (implied by any legal-adjacent system)	Confidence tiers + manual review queue (Part 8)
NOTE This handbook does not invent requirements beyond what the PS and the Rules support. Anything the team is tempted to add — blockchain, facial recognition, IoT sensors — must be checked against this table first. If it doesn't map to a requirement here, it goes on the Feature Kill List (Part 4).	

Part 3 — User Personas

Persona 1: Inspector Meena — Legal Metrology Field Inspector

Goals: Cover as many retail outlets/products as possible per day; build a defensible, evidence-backed case for any violation found.

Pain points today: Manually reading fine print under poor shop lighting; remembering every declaration requirement by heart; writing up findings by hand later.

Technical ability: Comfortable with a smartphone camera and basic apps; not a software person.

What LabelGuard should show them: One clear screen: upload/photo → instant flagged fields with images → one-tap “send for report.” Big text, minimal steps, works on a slow connection.

Persona 2: Supervisor Rakesh — Inspection Supervisor

Goals: Review flagged/uncertain cases from field inspectors; ensure consistency across inspectors; track team output.

Pain points today: No visibility into what inspectors actually checked; inconsistent documentation quality.

Technical ability: Comfortable with spreadsheets and basic dashboards.

What LabelGuard should show them: Inspection history list, filter by confidence/status, one-click drill-down into evidence for any case flagged as uncertain.

Persona 3: Administrative Officer Anjali — Compiles inspection statistics for the department

Goals: Produce monthly/periodic summaries of inspection activity and common violation types.

Pain points today: Manually compiling numbers from paper reports.

Technical ability: Comfortable with basic reporting tools.

What LabelGuard should show them: Aggregate counts and exportable reports — explicitly marked should-have/optional for the 5-day build, not MVP.

Persona 4 (future, optional): Manufacturer Self-Check User — Only if time permits — not core to SIH26034's inspector-facing framing

Goals: Check a label before printing to avoid a future violation.

Pain points today: Finding out about a labeling mistake only after enforcement.

Technical ability: Varies.

What LabelGuard should show them: The exact same evidence-first result screen, reframed as a pre-print self-check — explicitly optional (Part 4).

User Journey (primary persona)

10. Meena opens LabelGuard on her phone/laptop in the field.
11. She photographs the product's label (or uploads a photo taken moments before).
12. The system tells her within seconds if the image is usable.

13. OCR + extraction runs; she sees the declared fields laid out clearly.
14. The rule engine highlights any field it disagrees with, with an image region and confidence score.
15. For anything low-confidence, she is prompted to confirm manually.
16. She taps “Generate report” and the case is saved with a PDF she can file or forward to her supervisor.

Part 4 — The Exact MVP

MUST HAVE (the 5-day critical path)

17. Image upload
18. Image quality check (blur / glare / resolution gate)
19. OCR
20. Declaration extraction (regex/heuristic field parsing)
21. Structured data (a defined JSON schema for a package record)
22. Compliance rule engine
23. Violation detection with severity
24. Evidence mapping (bounding box ↔ finding)
25. Confidence score per finding
26. Human-review path for low-confidence/uncertain results
27. Inspection history (list + detail view)
28. Report generation (PDF)

SHOULD HAVE (only after MUST HAVE is fully working)

- Camera capture directly in-browser (not just file upload)
- Multilingual OCR (Hindi + regional language support)
- Dashboard analytics (counts by violation type)
- Richer evidence visualization (side-by-side zoom, annotated overlay toggle)

OPTIONAL (nice-to-have, cut first if behind schedule)

- LLM-assisted extraction/normalization on top of regex rules
- Semantic normalization of free-text address/manufacturer fields
- Advanced computer vision (perspective correction for curved packaging)
- Offline mode
- Mobile wrapper/PWA packaging

DO NOT BUILD

- Blockchain anything
- A custom-trained LLM
- A complicated chatbot interface
- IoT/sensor integration
- Facial recognition
- Real-time video surveillance
- A multi-service microservice architecture
- Multiple frontend applications
- Complex multi-role authentication (unless a judge explicitly requires role-based login — a single shared demo login is enough)

- Unnecessary cloud infrastructure (Kubernetes, multi-region, etc.)

FEATURE KILL LIST — what to cut, and in what order, if behind schedule

Cut Order	Feature to Drop	Why it's safe to drop
1st	Dashboard analytics	Purely cosmetic for the demo; history list is enough.
2nd	Multilingual OCR	English-label demo images are sufficient to prove the pipeline.
3rd	Camera capture in-browser	File upload achieves the same demo outcome with 1/10th the engineering risk.
4th	LLM-assisted normalization	Regex-only extraction on a controlled demo dataset still works; add LLM assist only if ahead of schedule.
5th	PDF styling polish	A plain but correct PDF report beats a beautiful one that crashes.
Never cut	Evidence mapping + confidence + human review	This is the entire USP (Part 7/8). If this goes, the project is a generic OCR demo.

Part 5 — Product Workflow

End-to-end pipeline (also see the architecture diagram in Part 53):

**USER → UPLOAD/CAMERA → IMAGE QUALITY CHECK → OCR → FIELD EXTRACTION → NORMALIZATION →
RULE ENGINE → COMPLIANCE RESULT → EVIDENCE → HUMAN REVIEW IF NECESSARY → REPORT → HISTORY**

Step	Input	Processing	Output	Failure case	Fallback	Demo note
1. Upload/Camera	Image file (jpg/png)	Validate file type + size; store temp copy	A usable image file in the backend's temp storage	Wrong file type, file too large, corrupted upload	Reject with a clear error and let the user retry	Show the upload happening live — audiences respond to seeing the raw photo appear.
2. Image Quality Check	Raw image	Blur detection (e.g. Laplacian variance), brightness/glare check, resolution check	PASS / RETAKE verdict with reason	Image too blurry, too dark, too small	Ask user to retake before wasting an OCR call	Deliberately demo a bad photo being rejected — this single moment proves the system isn't naive.
3. OCR	Quality-passed image	Text detection + recognition (PaddleOCR)	Raw text tokens + bounding boxes + per-token confidence	OCR returns empty, garbled, or wrong text	Retry with basic preprocessing (grayscale, contrast, denoise); fall	Show the OCR output overlay on the image —

Step	Input	Processing	Output	Failure case	Fallback	Demo note
					back to a second OCR pass	visually convincing.
4. Field Extraction	OCR tokens	Regex + keyword-position heuristics map tokens to fields (MRP, net qty, mfg date, address, etc.)	Partially structured field:value pairs with a source token/box per field	Field not found, wrong field matched	Mark as “not detected” rather than guessing	Show raw text turning into a clean structured table.
5. Normalization	Extracted fields	Standardize units (g/kg/ml/l), date formats, currency symbols	Clean, comparable values	Ambiguous unit, OCR noise inside a numeric field	Flag as low-confidence rather than silently “correcting”	Not usually demoed directly — it’s invisible plumbing that makes the rule engine reliable.
6. Rule Engine	Normalized fields	Evaluate each field against the applicable rule (Part 9/10)	Per-rule PASS / FAIL / UNCERTAIN + severity + confidence	Rule references a field that wasn't extracted	Auto-classify as UNCERTAIN, never FAIL, when a required field is simply missing from OCR (missing ≠ proven non-compliant)	This is the deterministic core — emphasize it is NOT an LLM decision.
7. Compliance Result	Rule outputs	Aggregate into an overall case status	Compliant / Potential Non-Compliance / Manual Review Required	Conflicting rule outputs	Most severe verdict wins, tie-break toward manual review	This is the headline screen — keep it visually

Step	Input	Processing	Output	Failure case	Fallback	Demo note
						simple.
8. Evidence	Rule outputs + original image	Map each finding to its source bounding box	Annotated image with highlighted regions per finding	Bounding box missing or misaligned	Show the field text instead of a box; never show a wrong box	This is the signature feature — spend the most polish here (Part 7).
9. Human Review	Low-confidence/uncertain findings	Route to a review queue; officer confirms/overrides	Officer-confirmed final status	Officer disagrees with the system	Officer's judgment always wins — log the override, don't block it	Demo one case being sent here to prove human-in-the-loop is real, not decorative.
10. Report	Final case	Render a structured PDF (ReportLab)	Downloadable inspection report	PDF generation error (missing font, broken image path)	Fall back to a plain-text/HTML report	End the demo on this artifact — it's the tangible deliverable a judge can hold.
11. History	Saved case	Persist to SQLite; list + detail views	Searchable inspection history	Database write failure	Keep an in-memory fallback list for the duration of the demo	Show a second, previously-completed case

Step	Input	Processing	Output	Failure case	Fallback	Demo note
						in history to prove persistence.

Part 6 — Product UI

12 screens, each with purpose, components, states. Text wireframes are illustrative layout guides for Member 2 — not pixel specs.

1. Demo Landing / Login

Purpose: Single shared demo login — no real auth system needed for MVP.

Components/fields/buttons: LabelGuard logo, tagline, “Start Inspection” button.

States: Default only — no loading/error states needed.

2. Dashboard

Purpose: Entry point after login; shows recent inspections + a “New Inspection” call to action.

Components/fields/buttons: Summary counters (optional/should-have), recent inspection list, New Inspection button.

States: Empty (no inspections yet), loaded, error (history failed to load).

3. New Inspection

Purpose: Start a case; capture basic context.

Components/fields/buttons: Product name/category free-text field (optional), Continue button.

States: Default only.

4. Image Upload

Purpose: Get the label photo into the system.

Components/fields/buttons: Drag-and-drop/file picker, thumbnail preview, Upload button.

States: Empty, file selected, uploading (progress), upload error.

5. Image Quality

Purpose: Confirm the photo is usable before spending an OCR call.

Components/fields/buttons: Quality verdict banner (GOOD/RETAKE), reason text, Retake/Continue buttons.

States: Checking (loading), GOOD, RETAKE with reason, error.

6. OCR Processing

Purpose: Show the system is actively reading the label — keeps user engaged during processing.

Components/fields/buttons: Progress indicator, live status text (“Reading label...”).

States: Loading, done (auto-advances), error/timeout with retry.

7. Extracted Declarations

Purpose: Show what the system read, structured, before judging it.

Components/fields/buttons: Table: Field name | Detected value | Source confidence.

States: Loaded, some fields “not detected,” error.

8. Compliance Result

Purpose: The headline verdict screen.

Components/fields/buttons: Overall status banner, list of findings with severity chips (Major/Minor).

States: Compliant (green), Potential Non-Compliance (red/amber), Manual Review Required (grey), loading, error.

9. Evidence View

Purpose: The signature feature — show WHY.

Components/fields/buttons: Original image with highlighted bounding boxes, tap-to-expand per finding, confidence %, rule reference.

States: Loaded, no evidence available for a finding (fallback to text-only), error.

10. Inspection History

Purpose: List of past cases.

Components/fields/buttons: Table: Date | Product | Status | Confidence — filter by status.

States: Empty, loaded, filtered-empty.

11. Inspection Details

Purpose: Full record for one past case.

Components/fields/buttons: All prior screens' data read-only + officer notes field.

States: Loaded, error.

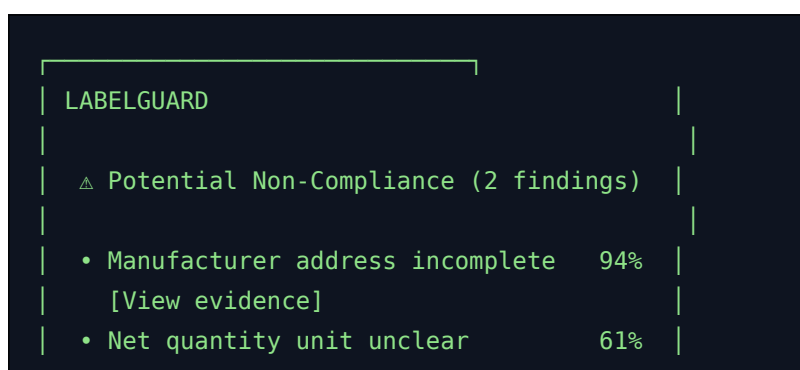
12. Report Generation

Purpose: Produce the artifact an inspector files.

Components/fields/buttons: Generate PDF button, download link, preview thumbnail.

States: Idle, generating, ready, error with retry.

Illustrative Wireframe — Compliance Result Screen





Part 7 — The Signature Feature: Evidence-First Compliance

The USP is not “we used AI.” It is that every single finding is traceable, end to end:

RULE → FIELD → DETECTED VALUE → VIOLATION → IMAGE REGION → CONFIDENCE → HUMAN REVIEW

Example finding card

Violation	Manufacturer address incomplete
Evidence	Bounding box around the manufacturer-details region of the label
Confidence	94%
Severity	Major
Action	Requires authorised human review before any formal notice is issued

Why this beats “AI says non-compliant”

- A bare AI verdict is unfalsifiable — the inspector can't check the system's work without redoing it from scratch.
- An evidence-linked verdict lets the inspector verify the finding in under two seconds by glancing at the highlighted region.
- It creates a defensible, auditable record: if a case is challenged later, the exact image evidence and confidence are already attached.
- It is honest about uncertainty instead of projecting false authority — which is what a legal-adjacent tool must do.

Part 8 — Human-in-the-Loop System

An LLM (or any single automated pipeline) must never be the final legal decision-maker on a regulatory question. LabelGuard enforces this with three confidence tiers:

Tier	Threshold (prototype default)	Behavior
HIGH confidence	≥ 90% field + rule confidence	Automatic preliminary result shown, still labeled “preliminary” — never “final.”
MEDIUM confidence	70–89%	Result shown, but flagged “Requires reviewer confirmation” before it can be included in a filed report.
LOW confidence	< 70%, or a required field wasn't detected at all	No automatic verdict. “Manual verification required” is the only output for that finding.

NOTE — Why this reduces risk

It prevents a bad OCR read (glare, curved packaging, tiny font) from becoming a false compliance claim.

It keeps a human legally and practically in charge of every consequential decision, which is exactly the posture regulators and judges expect from an AI-adjacent legal tool.

Thresholds are prototype defaults (Category C) — a real deployment would calibrate them against a validated labeled dataset (Part 24), which the team should say plainly if asked.

Part 9 — Legal Source of Truth

WARNING — Read this before writing a single rule

This project touches a real regulation. Do not let an LLM invent, simplify, or guess a legal requirement. Every rule implemented in the prototype must trace to one of the official sources below, and must be labeled with its source category from Part 2.

Official sources (verified live, 28 August 2026)

1. Department of Consumer Affairs — official page hosting the Legal Metrology (Packaged Commodities) Rules, 2011 (downloadable original notification): <https://consumeraffairs.nic.in/legalmetrologyactsandrules/legal-metrology-packaged-commodities-rules-2011>
2. Department of Consumer Affairs — consolidated Rules with all amendments, official PDF: https://consumeraffairs.gov.in/public/upload/admin/cmsfiles/whatsnews/Book_on_Legal_Metrology_Packaged_Commodities_Rules,2011_with_all_amendments_whatsnews.pdf
3. Department of Consumer Affairs — official Frequently Asked Questions on Legal Metrology, PDF: https://consumeraffairs.gov.in/public/upload/admin/cmsfiles/whatsnews/Frequently_Asked_Questions_on_Legal_Metrology_whatsnews.pdf
4. The Legal Metrology Act, 2009 (the parent Act under which the 2011 Rules were made) — hosted on the same Department of Consumer Affairs site.
5. The Legal Metrology (Packaged Commodities) (Amendment) Rules, 2023 — notified 6 October 2023, extending specific declaration requirements (manufacturer/importer/seller details, consumer care contact, MRP, net quantity) to e-commerce sales of loose/unpackaged commodities. Confirms the Rules are periodically amended; the team must record the version date it built against.

NOTE

The official FAQ PDF above (source 3) directly confirms two small but real implementation details worth encoding in the rule engine: (a) declaring the importer's name/address alone is sufficient where only an importer exists — not a defect; (b) the retail sale price may be declared using either the “₹” symbol or the words “Rs.” — both are valid, so the price-presence regex must accept either.

NOTE

Category note [A]: Items 1–5 above are official/primary sources. Everything the rule engine implements in Part 10 must cite one of these, with the exact rule number where possible, plus the version-date the team checked it against.

Core declaration requirements the MVP targets (from Rule 6 and related provisions of the 2011 Rules)

Declaration	What the Rules require (summarized, not verbatim — verify exact wording against source 1 above)	Machine-checkable from a photo?
Manufacturer/packer/importer name & address	Name and address of the manufacturer, or the packer and manufacturer, or the importer for an imported package, must appear on the label.	PARTIAL — OCR can detect presence and completeness heuristics (has a name-like line + address-like line), but cannot verify legal correctness of the entity.
Common/generic name of the commodity	The commodity's common or generic name must be declared, not only a brand name.	PARTIAL — presence check only.
Net quantity	Net quantity in standard units of weight/measure/number must be declared.	PARTIAL — presence + unit-format check; correctness of the underlying fill weight is NOT verifiable from an image.
Retail sale price (MRP)	MRP inclusive of all taxes, in Indian currency, must be declared. Per the official FAQ (source 3 below), either the “₹” symbol or the words “Rs.” are acceptable — both must be treated as valid.	PARTIAL — presence + currency-symbol/format check (accepting both ₹ and Rs.).
Date of manufacture/packing/import	A date declaration must appear.	PARTIAL — presence + date-format plausibility check.
Consumer care details	A means of contacting the manufacturer/packer for consumer complaints must be declared.	PARTIAL — presence check (phone/email pattern detection).
Country of origin (imported goods)	Country of origin must be declared for imported packages.	PARTIAL — presence check only, and only relevant if the package is identified as imported.

Per-rule specification format (fill this in as rules are implemented — see Part 10 for the schema)

RULE ID	RULE-LM-001
Source	Legal Metrology (Packaged Commodities) Rules, 2011, Rule 6 [A] — verify exact sub-clause against source 1, note check date
Requirement	Manufacturer/packer/importer name and address must be present on the label
Machine-checkable?	PARTIAL
Implementation	Presence heuristic: OCR text block matching a name-line followed by an address-like line (pincode

	pattern, city keyword)
Limitation	Cannot verify the declared entity is legally correct or currently registered; OCR failure on this region yields UNCERTAIN, never a false FAIL

 WARNING — Non-negotiable language rules for the whole product

Use: “potential non-compliance,” “inspection assistance,” “preliminary verification,” “requires authorised human review.”

Never say: “the AI legally certifies a package,” “LabelGuard proves this is illegal,” or anything implying the software itself makes a binding legal determination.

If a requirement cannot be reliably machine-checked from an image (e.g. verifying an MRP matches the actual fill weight, or that a registered entity exists), the correct system output is literally the words “Manual verification required” — not a best guess.

Part 10 — Rule Engine

The rule engine is the deterministic heart of LabelGuard. It is plain Python + JSON — no machine learning, no LLM call — so every decision can be explained and reproduced exactly.

Folder structure

```
rules/
├─ declarations.json      # manufacturer, common name, presence rules
├─ quantity.json          # net quantity format + unit rules
├─ price.json             # MRP format rules
├─ manufacturer.json      # name/address presence + completeness heuristics
├─ consumer_care.json     # phone/email pattern rules
├─ date.json              # manufacture/packing date format rules
├─ country_of_origin.json # import-only rule
└─ severity.py            # maps rule failures to Major/Minor
```

Rule schema (JSON)

```
{
  "rule_id": "RULE-LM-003",
  "field": "net_quantity",
  "source": "Legal Metrology (Packaged Commodities) Rules, 2011, Rule 6",
  "description": "Net quantity must be present and expressed in a standard unit",
  "check_type": "regex_presence",
  "pattern": "\\d+(\\.\\d+)?\\s?(g|kg|ml|l|gm|gms)\\b",
  "severity_if_fail": "MAJOR",
  "confidence_floor": 0.7,
  "on_field_missing": "UNCERTAIN"
}
```

Pseudocode: rule evaluation loop

```
def evaluate_case(extracted_fields, rules):
    results = []
    for rule in rules:
        field_value = extracted_fields.get(rule["field"])
        if field_value is None:
            results.append(Result(rule, status="UNCERTAIN",
                                reason="field not detected by OCR"))
            continue
        matched = re.search(rule["pattern"], field_value.text)
        confidence = field_value.confidence
        if confidence < rule["confidence_floor"]:
            status = "UNCERTAIN"
```

```
elif matched:
    status = "PASS"
else:
    status = "FAIL"
results.append(Result(rule, status, confidence,
                      field_value.bounding_box))
return results
```

Design rules for the engine

- Every rule is a small, testable, human-readable JSON object — never a hardcoded magic condition buried in application code.
- A missing field is UNCERTAIN, never FAIL. Absence of evidence is not evidence of a violation.
- Confidence floors are configurable per rule so Member 4 can tune sensitivity without touching code.
- Severity (Major/Minor) is assigned per rule based on how central the declaration is (e.g. missing MRP = Major; a minor formatting inconsistency = Minor).
- Explainability is not optional: every Result object must carry the rule_id, the source citation, and the exact bounding box so Part 7's evidence view has something to point at.

NOTE

Do not hardcode an uncertain or disputed legal interpretation as if it were settled. If the team is unsure whether a requirement is machine-checkable or how strictly to interpret it, mark it UNCERTAIN/manual-review by default rather than picking a strict reading that could misrepresent the law.

Part 11 — OCR / Computer Vision

OCR in one sentence

OCR (Optical Character Recognition) takes a photo and returns the text it can find in it, usually along with where on the image each piece of text was found (a bounding box) and how confident the engine is.

Engine comparison (researched, current as of mid-2026)

Engine	Strengths	Weaknesses	Verdict for this project
Tesseract	Free, mature, huge language support, good on clean/high-contrast text	Struggles with complex layouts and busy backgrounds; needs more manual preprocessing to do well	Solid fallback / second-opinion engine
PaddleOCR	Strong on multilingual text and messier real-world layouts; ships detection + recognition + orientation + confidence out of the box; fast	Heavier install (PaddlePaddle framework dependency); Windows setup can be fiddly for complete beginners	Recommended primary engine — label photos are exactly the messy real-world case it's built for
Cloud OCR APIs (Azure/AWS/Google)	Very high raw accuracy, minimal setup	Needs an API key + internet + can incur cost; creates a single point of failure during a live demo	Optional secondary path only, with an offline fallback for the demo (Part 38)
LLM vision (e.g. asking an LLM to read text in an image)	Can be surprisingly good at reading label text, and useful for messy semantic parsing	Not deterministic, no guaranteed bounding boxes, and must never be the compliance decision-maker (Part 12)	Optional assist for normalization/parsing only — never for the core OCR-to-rule-engine path

NOTE — Recommendation

Use PaddleOCR as the primary engine. Keep Tesseract installed as a fallback the team can switch to in 10 minutes if PaddleOCR setup blocks a specific machine (Part 45 has both install paths).

Preprocessing pipeline (in order)

29. Resize — normalize to a reasonable max dimension (e.g. 1600px longest side) so OCR runtime is predictable.
30. Grayscale conversion — removes color noise that doesn't help text recognition.
31. Contrast enhancement — helps with dim/underexposed photos.
32. Denoising — a light blur/median filter to reduce JPEG artifacts before thresholding.
33. (Optional, should-have) Perspective correction — for labels photographed at an angle.

Decision table

OCR works locally on your test images	Use it as-is. Don't over-engineer preprocessing you don't need yet.
OCR accuracy is poor on a specific image	Apply the preprocessing pipeline above, one step at a time, and re-test after each step.
Still poor after preprocessing	Try the second OCR engine (Tesseract) on the same image and compare.
Team is relying on an internet-dependent OCR API	Pre-build a small "known-good" demo dataset with cached OCR results so the live demo never depends on live internet access to that API.

What "good OCR output" looks like

A usable OCR result returns: (1) recognized text per line/word, (2) a bounding box (4 coordinates) per line/word, (3) a confidence score per line/word. If the OCR library used doesn't return all three, field extraction and evidence mapping (Parts 5, 7) cannot work — confirm this before committing to an engine.

Part 12 — AI / LLM Architecture (Where AI Belongs)

GOOD uses of AI/LLMs in this project	BAD uses of AI/LLMs in this project
OCR text cleanup (fixing obvious OCR typos in a detected field)	Making the final legal compliance decision
Field extraction assistance (helping parse messy address blocks)	Inventing a legal rule that isn't in Part 9's source table
Normalization (turning "500GM" and "0.5 KG" into a comparable value)	Making an unsupported compliance claim ("this package is definitely illegal")
Natural-language explanation of a rule-engine result for the UI	Replacing the deterministic rule engine entirely
Developer assistance: writing code, tests, docs, debugging (this entire handbook's Part 15–17)	Being the only thing standing between the system and a false accusation of non-compliance

Architecture diagram

IMAGE → OCR/CV → STRUCTURED DECLARATION EXTRACTION (regex first, LLM-assisted only for messy normalization) → DETERMINISTIC RULE ENGINE → COMPLIANCE RESULT → EVIDENCE → HUMAN REVIEW WHEN UNCERTAIN.

NOTE

This diagram is the single most important architecture fact in the whole handbook. If a judge asks “where does the AI make the compliance decision,” the correct answer is: it doesn't — the rule engine does, and the AI only assists the steps around it.

Part 13 — Exact Tech Stack

Layer	Choice	Why
Frontend	Next.js + React + Tailwind CSS	Huge amount of LLM training data exists on this exact combination, so Claude/ChatGPT/Cursor generate very reliable code for it; Tailwind avoids hand-writing CSS from scratch.
Backend	FastAPI (Python)	Minimal boilerplate, automatic interactive API docs, and the same language as the OCR/CV/rule-engine code — one language for the whole backend reduces context-switching for beginners.
Language	Python (backend, OCR, rules) + TypeScript/JavaScript (frontend)	Python has the richest OCR/CV ecosystem; Next.js expects TypeScript/JS.
Database	SQLite to start; PostgreSQL only if the team hits a real concurrency limit	SQLite is a single file, zero setup, perfect for a 5-day single-server prototype. Don't add PostgreSQL unless something SQLite can't do actually blocks you.
OCR	PaddleOCR (primary), Tesseract (fallback)	See Part 11.
Computer vision	OpenCV	Industry-standard, well documented, handles preprocessing and bounding-box drawing.
PDF generation	ReportLab	Mature, scriptable, doesn't require a headless browser.
Version control	Git + GitHub	Non-negotiable for a 6-person team; see Part 21.
Deployment	Simplest option that reliably demos (e.g. running locally on the presentation laptop, or a single free-tier cloud host as backup)	Do not add infrastructure complexity that isn't needed to show the demo working in the room.

Installation — Windows (PowerShell)

```
# 1. Node.js (for Next.js) – download LTS installer from nodejs.org, then verify:
node -v
npm -v

# 2. Python (3.10+) – download from python.org, check "Add to PATH" during install
python --version

# 3. Git – download from git-scm.com
git --version
```

```
# 4. Create project + virtual environment
mkdir labelguard && cd labelguard
python -m venv venv
.\venv\Scripts\Activate.ps1

# 5. Backend dependencies
pip install fastapi uvicorn python-multipart
pip install paddlepaddle
pip install paddleocr
pip install opencv-python reportlab

# 6. Frontend
npx create-next-app@latest frontend --typescript --tailwind --app
cd frontend && npm install
```

Installation — macOS/Linux

```
# 1. Node.js (via nvm is easiest)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
nvm install --lts

# 2. Python (macOS usually has python3 already; Linux: sudo apt install python3
python3-venv)
python3 --version

# 3. Git (macOS: usually preinstalled or via Xcode CLT; Linux: sudo apt install
git)
git --version

# 4. Project + virtual environment
mkdir labelguard && cd labelguard
python3 -m venv venv
source venv/bin/activate

# 5. Backend dependencies
pip install fastapi uvicorn python-multipart
pip install paddlepaddle
pip install paddleocr
pip install opencv-python reportlab

# 6. Frontend
npx create-next-app@latest frontend --typescript --tailwind --app
cd frontend && npm install
```

⚠ WARNING — If PaddleOCR install fights you on Windows

This is a known friction point (shapely/C-extension build issues on some Windows setups). Don't burn more than 30 minutes on it during the hackathon — switch to `pip install pytesseract` plus the Tesseract binary from the official UB-Mannheim Windows build, and use the Part 11 fallback path instead. A working fallback engine beats a broken primary one.

Part 14 — Official Learning Resources

A short, curated curriculum — not a link dump. Prioritized so nobody drowns.

Resource	What to learn	Est. time	Who	Day	Priority
Next.js Learn (nextjs.org/learn)	App Router basics, pages, data fetching	2–3 hrs	Member 2	Day 1	MUST READ
FastAPI official tutorial (fastapi.tiangolo.com/tutorial)	Path operations, request bodies, file uploads	1.5–2 hrs	Members 1, 5	Day 1	MUST READ
Python official docs — tutorial section (docs.python.org/3/tutorial)	Only if genuinely new to Python syntax	as needed	All	Day 1	READ IF BLOCKED
Pro Git book, Ch. 2–3 (git-scm.com/book)	Basic Git workflow	1 hr	All	Day 1	MUST READ
OpenCV-Python tutorials (docs.opencv.org)	Reading images, basic preprocessing (grayscale, blur, threshold)	1.5 hrs	Member 3	Day 1–2	MUST READ
PaddleOCR quickstart (github.com/PaddlePaddle/PaddleOCR)	Install, run inference, read output format	1 hr	Member 3	Day 1–2	MUST READ
SQLite official docs (sqlite.org/docs.html)	Basic schema + queries	45 min	Member 5	Day 1	MUST READ
Tailwind CSS docs (tailwindcss.com/docs)	Utility classes for layout/spacing/color	as needed	Member 2	Day 1–2	READ IF BLOCKED
GitHub docs — Pull Requests (docs.github.com)	PR + review flow	30 min	Member 1	Day 1	MUST READ
ReportLab user guide	Building a simple PDF from Python	1 hr	Member 5	Day 4	READ IF BLOCKED

NOTE

Everything not marked MUST READ or READ IF BLOCKED is OPTIONAL. Do not read documentation for its own sake during a 5-day sprint — read exactly enough to unblock the next task, then get back to building.

Part 15 — LLM Tool Strategy

WARNING — Verify before you commit

AI tool capabilities, pricing, and free-tier limits change frequently. The comparisons below reflect research at the time this handbook was written. Before the hackathon starts, have Member 1 spend 15 minutes re-checking each tool's current pricing page — do not assume anything below is still exactly accurate.

What the current landscape looks like

- Terminal/CLI-native coding agents (e.g. Claude Code) tend to lead on complex, multi-file changes and long, autonomous tasks — useful for scaffolding and larger refactors.
- IDE-native agents (e.g. Cursor) are strong for fast, in-editor, file-by-file iteration with visible diffs — useful for day-to-day feature work.
- Chat-based general assistants (e.g. ChatGPT, Gemini, and Claude's own chat interface) are strong for research, architecture discussion, explaining errors in plain language, and drafting documentation/pitch material.
- Long-context tools are useful when you need to paste a large chunk of the codebase or a long log file at once for debugging.
- Free tiers exist across most of these but usually cap out fast under heavy daily use — assume the team will want at least one paid seat for the 5 days if the budget allows, but design the workflow so no single subsystem breaks if one tool becomes unavailable mid-hackathon.

Best-for matrix

Task	Primary pick	Backup pick	Notes
Architecture / planning	A chat-based assistant with a large context window	Any second chat assistant for a sanity-check opinion	Use for the PROJECT_CONTEXT.md and ARCHITECTURE.md docs in Part 41 before writing code.
Coding (scaffolding, multi-file features)	A terminal/IDE coding agent	Chat assistant + manual copy-paste	Best when it can see the whole repo, not just a pasted snippet.
Debugging	Whichever assistant you're already in — paste the exact error + relevant code	A second model as a fresh pair of eyes if the first is stuck	See the exact debugging prompt template in Part 17.
UI generation	IDE coding agent or chat assistant with the frontend-design constraints in hand	—	Give it the exact Tailwind/Next.js version and any component library in use.
Research (regulations, competitors, libraries)	A chat assistant with live web search enabled	—	Never trust an answer about current regulation or pricing without a live search

Task	Primary pick	Backup pick	Notes
			— see Part 9.
Documentation / pitch writing	Any chat assistant	—	Good for tightening language, not for inventing facts — always fact-check against Part 9/29.
Testing	IDE/terminal coding agent (writes + runs tests in place)	Chat assistant (drafts test cases to paste in manually)	—

Recommended hierarchy for this team

PRIMARY CODING LLM	A strong terminal- or IDE-native coding agent the team has access to (e.g. Claude Code, or an IDE agent like Cursor) — whichever the team can get seats for. Use it for scaffolding and any change touching more than one file.
SECONDARY DEBUGGING LLM	A general chat assistant for pasting a single error + relevant snippet when the primary agent is stuck or a fresh perspective is needed.
RESEARCH LLM	Whichever assistant has reliable live web search — use for Part 9 (legal source verification), Part 29 (competitors), and any library/tool fact that could be stale.
UI ASSISTANT	The same primary coding agent, given the constraints from Part 6 (screens/states) explicitly in the prompt.
DOCUMENTATION ASSISTANT	A general chat assistant — low-risk, high-value use for README, pitch deck text tightening, and comments.

NOTE — Never single-point-of-fail on one paid API

Pick a primary tool, but keep at least one free-tier chat assistant as backup in case a paid seat runs out, a service has an outage, or a teammate's account hits a rate limit mid-hackathon. Losing your only LLM access on Day 3 must not be able to stop the whole team.

Part 16 — The Golden LLM Prompting System

Beginners get bad results from LLMs almost always for the same reason: too little context, too vague an ask. This section fixes that with one reusable master prompt plus a copy-paste prompt for every core task.

The Master Developer Prompt (paste this at the start of every new LLM session/chat for this project)

♦ COPY-PASTE PROMPT — Master Context Prompt

You are my senior engineer for a 5-day hackathon project called LabelGuard.

PROJECT: A web app that lets an inspector upload/photograph a packaged product label, runs OCR to read it, extracts structured declarations (manufacturer, net quantity, MRP, date, consumer care, country of origin), checks them against an explicit rule engine (NOT an LLM decision), highlights evidence on the image for any flagged issue, and generates a PDF inspection report. Legal basis: India's Legal Metrology (Packaged Commodities) Rules, 2011.

STACK: Next.js + React + Tailwind CSS (frontend), FastAPI + Python (backend), SQLite (database), PaddleOCR (OCR), OpenCV (image preprocessing), ReportLab (PDF).

TEAM: 6 hardware engineers, almost no software background, 5 days total.

RULES FOR YOU:

1. Explain any new concept in one plain sentence before using it.
2. Give me the smallest working version first, not the most "complete" one.
3. Never make the AI the final compliance decision-maker — that is the rule engine's job.
4. Flag any place you are guessing instead of being sure.
5. Always give me a way to test what you just wrote.

I will now give you a specific task.

Copy-paste prompts for the 17 core tasks

♦ COPY-PASTE PROMPT — 1. Project scaffolding

Using the context above, scaffold the initial folder structure for LabelGuard: a /frontend (Next.js) and /backend (FastAPI) folder, a minimal working "hello world" route on each, and a top-level README explaining how to run both. Give me exact commands to run, in order, and how to verify each one worked before moving to the next.

◆ COPY-PASTE PROMPT — 2. Frontend page

Build the 'Image Upload' page from my UI spec: a drag-and-drop file input, a thumbnail preview once a file is chosen, and an 'Upload' button that POSTs the file as multipart/form-data to `http://localhost:8000/scan`. Include a loading state while uploading and an error state if the request fails. Use Tailwind for styling. Explain what multipart/form-data means in one sentence.

◆ COPY-PASTE PROMPT — 3. Backend endpoint

Create a FastAPI POST endpoint `/scan` that accepts an uploaded image file, saves it to a temp folder, and returns a JSON response with a generated `case_id` and the saved file path. Include basic validation: reject non-image file types and files over 10MB, with clear error messages. Give me a curl command to test this endpoint without the frontend.

◆ COPY-PASTE PROMPT — 4. OCR integration

Write a Python function that takes an image file path, runs it through PaddleOCR, and returns a list of `{text, bounding_box, confidence}` objects. Include a fallback: if PaddleOCR raises an exception, log it and return an empty list rather than crashing. Show me how to test this function on a single sample image from the command line.

◆ COPY-PASTE PROMPT — 5. Rule engine

Implement a rule evaluation function that takes a dict of extracted fields and a list of rule JSON objects (schema in Part 10) and returns a list of Result objects with `rule_id`, `status` (PASS/FAIL/UNCERTAIN), `confidence`, and `bounding_box`. A missing field must always produce UNCERTAIN, never FAIL. Write 3 unit tests: one PASS case, one FAIL case, one UNCERTAIN case.

◆ COPY-PASTE PROMPT — 6. Database

Design a SQLite schema for an 'inspections' table storing: `id`, `created_at`, `image_path`, `extracted_fields` (JSON text), `rule_results` (JSON text), `overall_status`, `officer_note`. Give me the CREATE TABLE statement and simple Python functions to insert a new inspection and fetch the most recent 20.

◆ COPY-PASTE PROMPT — 7. PDF generation

Write a Python function using ReportLab that takes one inspection record (fields + rule results) and produces a one-page PDF report with: a title, the product image, a table of extracted fields, and a list of findings with severity and confidence. Show me how to open and check the generated PDF.

◆ COPY-PASTE PROMPT — 8. Evidence highlighting

Write a function that takes the original image and a list of bounding boxes

(one per finding) and draws colored rectangles + labels on a copy of the image using OpenCV, returning the new image path. Use red for FAIL findings and yellow for UNCERTAIN. Show me how to preview the result.

♦ **COPY-PASTE PROMPT — 9. Debugging**

Here is an error I'm getting: [PASTE FULL ERROR]. Here is the relevant code: [PASTE CODE]. Here is what I expected to happen vs what actually happened. Explain the likely cause in plain language, then give me the smallest possible fix, then tell me exactly how to verify the fix worked.

♦ **COPY-PASTE PROMPT — 10. Refactoring**

This function works but I think it's messy: [PASTE CODE]. Refactor it for readability without changing its behavior. Show me a diff-style before/after and explain each change in one line.

♦ **COPY-PASTE PROMPT — 11. Testing**

Write pytest test cases for this rule engine function covering: a fully compliant case, a case missing one required field, a case with a low-confidence OCR read, and a case with conflicting data. Include comments explaining what each test is checking.

♦ **COPY-PASTE PROMPT — 12. UI improvement**

Here is a screenshot/description of my current UI: [DESCRIBE]. It needs to communicate 'this field has a compliance issue' clearly to a non-technical inspector glancing at a phone screen. Suggest 2 specific layout/color changes and give me the exact Tailwind classes to make them.

♦ **COPY-PASTE PROMPT — 13. Git commit**

I made these changes: [LIST FILES/CHANGES]. Write a clear, conventional-commit-style commit message (type: short summary, then 1-2 bullet points if needed) that a teammate could understand without opening the diff.

♦ **COPY-PASTE PROMPT — 14. Code review**

Review this code as if you were a strict but fair senior engineer: [PASTE CODE]. Check for: unhandled errors, hardcoded secrets, unclear naming, missing input validation. List issues by severity, don't just praise it.

♦ **COPY-PASTE PROMPT — 15. Security review**

Review this file-upload endpoint for basic security issues appropriate for a 5-day prototype: [PASTE CODE]. Check file type validation, file size limits, path traversal risk, and whether any API keys or secrets are hardcoded instead of using environment variables.

◆ COPY-PASTE PROMPT — 16. Deployment

I need the simplest possible way to deploy this FastAPI + Next.js app so I can demo it from a laptop and have a backup URL in case the laptop's Wi-Fi fails. Given my stack, what's the lowest-effort reliable option, and what are the exact steps?

◆ COPY-PASTE PROMPT — 17. Documentation

Write a short README section explaining how to install and run this project locally, assuming the reader has never used FastAPI or Next.js before. Include the exact commands and what each one does in one line.

Part 17 — LLM Debugging Playbook

For every common error: symptom → likely cause → what to check → prompt to give the LLM → manual fix → when to stop.

Symptom	Likely cause	What to check	Manual fix	When to stop / escalate
npm install failure	Node version mismatch, corrupted cache, or conflicting lockfile	Run <code>`node -v`</code> ; delete <code>node_modules + package-lock.json</code> ; retry	<code>`npm cache clean --force`</code> then reinstall	If it still fails after a clean install + Node LTS, switch machines rather than losing more time
Python dependency failure	Missing system build tools, wrong Python version, conflicting packages	Check <code>`python --version`</code> ; try installing inside a fresh venv	Recreate the virtual environment from scratch	After 2 clean-venv attempts, isolate the one offending package and search its GitHub issues directly
CORS error in browser console	Backend not sending the right Access-Control headers for the frontend's origin	Confirm frontend URL/port matches what backend CORS config allows	Add FastAPI's <code>CORSMiddleware</code> with the frontend's exact origin	N/A — this is always fixable in under 15 minutes
API returns 404	Wrong URL path, route not registered, or backend not running	Check the exact path in browser dev tools Network tab vs the FastAPI route decorator	Fix the mismatched path string	N/A
API returns 500	Unhandled exception inside the endpoint	Read the backend terminal's full traceback, not just the frontend error	Wrap the failing line in <code>try/except</code> and log the real error	If the traceback points inside a third-party library, check that library's docs/issues for the exact error text
JSON parsing failure	Backend returned something that isn't valid JSON (often an	Print the raw response body before calling <code>.json()</code>	Fix the backend to always return JSON, even on error	N/A

Symptom	Likely cause	What to check	Manual fix	When to stop / escalate
	HTML error page)			
OCR returns empty	Image too low quality, wrong file path, or OCR engine failed silently	Test the OCR function directly on a known-good sample image outside the full pipeline	Add the Part 11 preprocessing steps one at a time	If empty on 3+ different clean images, the install itself is broken — reinstall or switch to Tesseract fallback
OCR reads wrong text	Blur, glare, low resolution, or curved packaging	Look at the raw image quality first before blaming the library	Apply preprocessing (Part 11); retake photo with better lighting	If a specific demo image never reads correctly after 30 minutes, replace it with a cleaner backup demo image
Image upload fails	File size/type rejected, or multipart form mismatch between frontend and backend	Compare the exact request payload (Network tab) against what the backend endpoint expects	Align field name in FormData with the backend parameter name exactly	N/A
Frontend cannot reach backend	Wrong port, backend not running, or firewall/localhost mismatch	Confirm backend is running and note its exact port; check frontend's API base URL	Hardcode the correct localhost:PORT during development	N/A
Environment variable missing	`.env` file not loaded or not created from <code>`.env.example`</code>	Print the variable at startup to confirm it loaded	Create/fix the <code>`.env`</code> file; restart the server (env vars load at startup only)	N/A
Database error	Schema mismatch, locked file, or missing table	Open the SQLite file with a DB browser tool and inspect the schema directly	Re-run the CREATE TABLE migration; delete and recreate the dev database if data isn't precious yet	N/A
Git merge conflict	Two people edited the same lines	Open the conflicted file and read both <code>`<<<<<<`/`>>>>>>`</code> sections	Manually choose/combine the correct lines, remove the markers, then commit	If a conflict is too tangled to reason about, call Member 1

Symptom	Likely cause	What to check	Manual fix	When to stop / escalate
				before guessing
Deployment failure	Missing environment variable on the host, or a dependency present locally but not declared	Check the deployment platform's build/runtime logs, not just "it failed"	Add the missing dependency to requirements.txt/package.json explicitly	If deployment eats more than 2 hours on Day 4/5, fall back to local-laptop demo + Part 38's offline mode
PDF generation error	Missing font file, broken image path, or a None value passed where text was expected	Print every value going into the PDF function right before it's used	Add default/placeholder values for any field that might be missing	N/A
Bounding boxes misaligned	Coordinates from a resized image applied to the original-resolution image	Confirm which image size the coordinates were computed against	Scale bounding box coordinates by the resize ratio before drawing	N/A

Universal debugging prompt template

♦ COPY-PASTE PROMPT — Universal Debug Prompt

I am using [Next.js App Router with TypeScript / FastAPI with Python – pick one]. [Describe what you expected to happen]. Instead, [describe exactly what happened, paste the FULL error text/traceback]. Here is the relevant frontend code: [PASTE]. Here is the relevant backend code: [PASTE]. Here is the exact request/response I saw in the browser Network tab: [PASTE]. Identify the mismatch, explain it in beginner language, give me the smallest possible fix, and give me an exact command or step to test that the fix worked.

Part 18 — Member-by-Member Handbook

Six members, five days each. Every day below is broken into Morning / Afternoon / Evening, and the single most technically load-bearing task of each day gets a full detailed breakout (objective, prompt, test, definition of done, common failure, fallback) so anyone can look up e.g. “Day 2 → Member 4 → blocker” and find exactly what to do.

Member 1 — Tech Lead / Integrator

Mission: own the repository, the architecture, the API contract between frontend and backend, and the final end-to-end integration. Member 1 does not write every line of the pipeline — they make sure everyone else's pieces actually connect.

Day	Morning	Afternoon	Evening
1	Set up GitHub repo, branch strategy (Part 21), folder skeleton (Part 53)	Scaffold Next.js frontend + FastAPI backend “hello world”; confirm frontend can call backend	Write PROJECT_CONTEXT.md and ARCHITECTURE.md (Part 41); run Day 1 evening review (Part 20)
2	Draft API_CONTRACT.md defining /scan, /extract endpoints with Member 5	Integrate Member 3's OCR function behind the /extract endpoint	Verify OCR results flow from backend to a raw JSON view in the frontend
3	Extend API_CONTRACT.md for /evaluate (rule engine) endpoint with Member 4 and 5	Wire rule engine output into the API response end-to-end	Resolve first real merge conflicts; run full standup + blocker meeting (Part 20)
4	Integrate evidence image generation (Member 3) and PDF report (Member 5) into one pipeline call	Build the single end-to-end “upload → report” happy-path flow	Run the full pipeline on all 3 demo packages; log every break in KNOWN_ISSUES.md
5	Feature freeze: no new features accepted after 12:00 (Part 55)	Bug triage across all members' modules; finalize deployment/backup plan (Part 38)	Final full run-through with the whole team; lock the demo script

Detailed breakout — Day 1: Repo + Skeleton Integration

Objective	Have a working GitHub repo where the frontend and backend can already talk to each other, even with placeholder content.
What to learn first	What a REST API call is (frontend sends a request, backend sends back JSON) — one sentence: it's just your two programs passing structured messages over HTTP.
What to build	A Next.js app with one page that calls a FastAPI endpoint returning {"status": "ok"} and displays it on screen.
Expected output	Running `npm run dev` in /frontend and `uvicorn main:app --reload` in /backend, then opening the frontend in a browser, shows the text “ok” somewhere on the page.
How to test	Open the browser at localhost:3000 and visually confirm the status text appears; also hit

	localhost:8000/health directly in the browser to confirm the raw JSON.
Definition of done	Both servers start with a single documented command each, and the frontend visibly displays data it received from the backend.
Common failure	CORS error in the browser console when the frontend calls the backend.
Fallback if it still fails	Use the exact CORS debugging row in Part 17; if still stuck after 20 minutes, temporarily hardcode <code>`allow_origins=["*"]`</code> in FastAPI's <code>CORSMiddleware</code> to unblock (tighten before Day 5 security pass, Part 26).

♦ COPY-PASTE PROMPT — Day 1 — Repo + Skeleton Integration

Using the context above, scaffold the initial folder structure for LabelGuard: a `/frontend` (Next.js, App Router, TypeScript, Tailwind) and `/backend` (FastAPI) folder at the repo root. Add a `GET /health` endpoint on the backend returning `{"status": "ok"}`. Add a page on the frontend that calls it on load and displays the status. Give me the exact commands to run both servers locally, in order, and how I'll know each one worked.

Detailed breakout — Day 3: End-to-End Compliance Result Wiring

Objective	A single API call takes an uploaded image and returns a full compliance result (not just raw OCR text).
What to learn first	The idea of a pipeline: each function's output becomes the next function's input, in a fixed order.
What to build	An orchestration function in the backend that calls: quality check → OCR → extraction → rule engine → assembles one JSON response.
Expected output	One JSON response containing extracted fields, rule results, and an <code>overall_status</code> .
How to test	<code>`curl -F "file=@sample.jpg" http://localhost:8000/inspect`</code> returns a complete JSON object, not an error.
Definition of done	The same endpoint works on all 3 demo images without crashing, even the deliberately low-quality one (which should return a manual-review result, not an error).
Common failure	500 error because one stage returns <code>None</code> and the next stage doesn't expect that.
Fallback if it still fails	Add an explicit <code>None</code> -check after every stage per the Part 17 500-error row; if a specific stage keeps breaking under time pressure, hardcode a safe default for that stage and flag it in <code>KNOWN_ISSUES.md</code> rather than blocking the whole team.

♦ COPY-PASTE PROMPT — Day 3 — End-to-End Compliance Result Wiring

I have these working pieces: an OCR function returning `[{text, bounding_box, confidence}]`, and a rule-evaluation function taking extracted fields and returning `[{rule_id, status, confidence, bounding_box}]`. Write one FastAPI endpoint `POST /inspect` that: accepts an image file, runs it through image quality check, then OCR, then a field-extraction step, then the rule engine,

and returns a single JSON object with all of it. If any stage fails, return a clear partial result instead of crashing. Show me how to test this with curl using a sample image.

Member 2 — Frontend Engineer

Mission: build all 12 screens from Part 6, wired to the real backend by Day 4, with every state (loading/empty/error/success) actually implemented — not just the happy path.

Day	Morning	Afternoon	Evening
1	Next.js Learn (Part 14); set up Tailwind	Scaffold all 12 screens as routes with placeholder content	Build the Image Upload screen UI (no backend yet)
2	Wire Image Upload to POST /scan; build Image Quality screen	Build OCR Processing (loading) screen with live status text	Test the upload → quality → processing flow manually
3	Build Extracted Declarations screen wired to real data	Build Compliance Result screen (the headline verdict UI)	Polish severity chips/colors; test with all 3 demo packages
4	Build Evidence View with bounding-box overlay image	Build Report Generation + Inspection History screens	Wire Inspection Details screen; full click-through test
5	Implement every missing loading/empty/error state (Part 47 DoD)	Responsive/demo-screen check on the actual presentation device	Rehearse driving the live demo click-by-click with Member 6

Detailed breakout — Day 1: Image Upload Screen

Objective	A working upload screen: pick a file, see a preview, click Upload.
What to learn first	React state (`useState`) is just a variable that, when changed, makes the screen re-render automatically.
What to build	A page with a file input, a thumbnail preview once a file is chosen, and an Upload button (backend call comes Day 2).
Expected output	A page where choosing an image shows its thumbnail, and clicking Upload logs the file object to the console.
How to test	Manually select 2-3 different image files and confirm the correct thumbnail appears each time.
Definition of done	Works with drag-and-drop AND click-to-browse; shows a thumbnail; button is disabled with no file selected.
Common failure	Thumbnail doesn't update when a new file is chosen.
Fallback if it still fails	Ask the LLM to check the `key` prop / whether the image URL is being regenerated with `URL.createObjectURL` on every new selection; if still stuck after 20 min, use a controlled `

♦ COPY-PASTE PROMPT — Day 1 — Image Upload Screen

Using Next.js App Router, TypeScript, and Tailwind CSS, build an Image Upload page component. Requirements: a drag-and-drop area that also supports click-to-browse, a thumbnail preview of the selected image before upload, and an 'Upload' button that is disabled until a file is chosen. For now, just `console.log` the file on button click – the real upload comes later. Explain what ``useState`` is doing in this component in one sentence.

Detailed breakout — Day 3: Compliance Result Screen

Objective	The headline verdict screen — the single most-viewed screen in the whole demo.
What to learn first	Conditional rendering: showing different UI depending on a status value (e.g. 'PASS' vs 'FAIL' vs 'UNCERTAIN').
What to build	A component that takes the <code>/inspect</code> API response and renders an overall status banner plus a list of findings with severity chips.
Expected output	A visually clear result screen matching the wireframe in Part 6, driven entirely by the props passed in.
How to test	Pass in 3 hardcoded example prop objects (compliant / non-compliant / manual-review) and confirm each renders correctly before wiring to the real API.
Definition of done	All three <code>overall_status</code> values render distinctly and correctly; clicking a finding fires the callback.
Common failure	Severity chip colors don't match across findings, or the layout breaks with 0 findings (compliant case).
Fallback if it still fails	Explicitly test the zero-findings case first — it's the one beginners forget; add an empty-state message ('No issues found') for that case.

♦ COPY-PASTE PROMPT — Day 3 — Compliance Result Screen

Build a Compliance Result component in Next.js + Tailwind that takes a prop shaped like `{ overall_status: 'compliant' | 'non_compliant' | 'manual_review', findings: [{ rule_id, field, status, severity, confidence, description }] }`. Show a colored banner for `overall_status` (green/red/grey) and a card per finding with a severity chip (Major=red, Minor=amber) and confidence shown as a percentage. Each finding card should be clickable – clicking it should call a prop function ``onViewEvidence(finding)``. Keep it readable on a small screen (mobile-first).

Member 3 — OCR / Computer Vision Engineer

Mission: turn a photo into reliable text + bounding boxes + confidence, and turn a rule-engine result back into a highlighted image. This is the most experimentation-heavy role — expect to try things and throw them away.

Day	Morning	Afternoon	Evening
1	Install OpenCV + PaddleOCR (Part 13); learn OpenCV basics	Write a standalone script: load image → run OCR → print results	Test on 5–10 real product photos taken on a phone
2	Build the image quality check (blur/glare/resolution gate)	Wrap OCR into a clean, reusable Python function	Hand the function to Member 5 to expose via API; test together
3	Add preprocessing steps (grayscale, contrast, denoise) per Part 11	Start the bounding-box drawing function for evidence	Compare OCR accuracy before/after preprocessing on the demo set
4	Finish evidence highlighting: color-coded boxes by severity	Test evidence drawing against real rule-engine output (with Member 4)	Full pipeline test on all 3 demo packages
5	Test everything under actual presentation lighting/device	Fix any last-mile visual bugs (misaligned boxes, wrong colors)	Freeze; support final rehearsal

Detailed breakout — Day 1: First Working OCR Script

Objective	Prove OCR works on your machine, on a real product photo, before building anything around it.
What to learn first	OCR returns three things you'll use everywhere: the text it found, where it found it (a box), and how sure it is (confidence 0–1).
What to build	A single Python script that loads one image and prints PaddleOCR's raw output.
Expected output	A printed list of lines like: 'MRP: Rs 45.00 confidence: 0.97 box: [[x,y],...]' for each piece of text found.
How to test	Run it on 3 different photos: one clean label, one blurry photo, one photo of plain text on paper — confirm results look reasonable on all three.
Definition of done	Runs without crashing on at least 5 different real product photos and returns non-empty, mostly-correct text on the clean ones.
Common failure	Import error or the model download hangs on first run.
Fallback if it still fails	Use the OCR debugging rows in Part 17; if PaddleOCR install is fighting the machine after 30 minutes, switch to the Tesseract fallback path (Part 11/13) and keep moving — don't lose a whole morning to one library.

♦ COPY-PASTE PROMPT — Day 1 — First Working OCR Script

Write a minimal Python script using PaddleOCR that: loads an image from a given file path, runs OCR on it, and prints each detected text line along with its bounding box coordinates and confidence score in a clean, readable format. Include the exact pip install commands needed first, and tell me what output I should expect to see if it's working correctly.

Detailed breakout — Day 2: Image Quality Gate

Objective	Reject unusable photos before wasting an OCR call on them — this is also a great demo moment (Part 5, Step 2).
What to learn first	A blurry image has less 'edge detail' than a sharp one — a simple statistical measure (Laplacian variance) can estimate this automatically.
What to build	A function <code>check_image_quality(image) -> {passed: bool, reason: str}</code> checking blur, brightness, and resolution.
Expected output	The function returns <code>passed=True</code> on a clear photo and <code>passed=False</code> with a specific reason on a blurry or dark one.
How to test	Deliberately take one blurry photo and one very dark photo; confirm both are correctly rejected with the right reason.
Definition of done	Correctly distinguishes at least 4 good photos from 4 deliberately bad ones (blurry, dark, or too small) in your test set.
Common failure	Thresholds reject good photos too, or accept genuinely bad ones.
Fallback if it still fails	Tune thresholds against your own demo dataset rather than trusting a generic default; if tuning eats too much time, widen the thresholds so it only catches extreme cases — false rejects are worse for a live demo than an occasional false accept.

♦ COPY-PASTE PROMPT — Day 2 — Image Quality Gate

Write a Python function using OpenCV that takes an image file path and returns a quality verdict: `{passed: bool, reason: str | None}`. Check three things: (1) blur, using Laplacian variance below a threshold means too blurry; (2) brightness, using mean pixel value too low means too dark; (3) resolution, below a minimum width/height means too small. Pick reasonable starting thresholds and explain how I should tune them using my own test images. Give me a quick way to test this on both a good and a deliberately bad (blurry/dark) photo.

Member 4 — Compliance Rule Engine Engineer

Mission: turn Part 9's legal source table into a working, tested, explainable rule engine. This role carries the most legal-accuracy responsibility on the team — when in doubt, mark UNCERTAIN, don't guess.

Day	Morning	Afternoon	Evening
1	Deep-read Part 9; learn basic regex	Draft the field list + JSON rule schema (Part 10)	Write the first draft of <code>declarations.json</code> with Member 6
2	Implement 3 rules (net quantity, MRP, manufacturer presence) with unit tests	Define the 3 confidence tiers (Part 8) in code	Test rules against Member 3's real OCR output for the first time

Day	Morning	Afternoon	Evening
3	Complete all 7 rule categories from Part 9	Build the full <code>evaluate_case()</code> aggregation function	Wire severity assignment; test with Member 1's /evaluate endpoint
4	Tune confidence thresholds against the real demo dataset	Add plain-language explanation text per rule result (for the UI)	Full regression test with Member 6's QA pass
5	Freeze rules — no logic changes after mid-morning	Fix any last bugs found in final QA	Be ready to explain any single rule's logic instantly for judge Q&A

Detailed breakout — Day 2: First 3 Rules + Regex (the classic Day 2 blocker)

Objective	Get the very first rule (net quantity presence + format) working end-to-end on a real extracted field.
What to learn first	Regex is a pattern-matching mini-language: <code>`d+`</code> means 'one or more digits', <code>`(g kg ml l)`</code> means 'one of these exact words'.
What to build	A JSON rule object for net quantity plus a Python function that checks a field's text against it.
Expected output	A regex pattern plus a short list of test strings with expected True/False results.
How to test	Run <code>`re.search(pattern, test_string)`</code> on each of the 5 test strings in a plain Python shell and confirm the match/no-match result is correct before touching the rule engine at all.
Definition of done	The regex correctly matches all 3 positive test strings and correctly rejects both negative ones, tested in isolation.
Common failure	REGEX NOT WORKING — pattern matches too much (false positives) or too little (misses valid formats like '500gm' or '0.5kg').
Fallback if it still fails	Test the regex in complete isolation (a plain Python REPL, not inside the app) before assuming the rule engine is broken — 90% of 'regex not working' bugs are the pattern itself, not the integration. If a specific unit variant keeps failing, add it explicitly to the pattern's alternatives rather than trying to write one 'clever' pattern that covers everything; a slightly longer, more literal regex beats a short broken one.

♦ COPY-PASTE PROMPT — Day 2 — First 3 Rules + Regex (the classic Day 2 blocker)

I'm writing a regex to detect a net-quantity declaration like '500g', '1.5 kg', '250 ml', or '1 l' inside a string that may have extra OCR noise around it (e.g. 'Net Qty: 500g e'). Write a Python regex pattern that matches the number+unit combination even with surrounding text, explain what each part of the pattern does in plain language, and give me 5 test strings (3 that should match, 2 that should not) so I can verify it with Python's ``re`` module before wiring it into the rule engine.

Detailed breakout — Day 3: Full `evaluate_case()` Aggregation

Objective	Combine every individual rule result into one overall case status, with a clear tie-break rule.
What to learn first	Aggregation logic: given several PASS/FAIL/UNCERTAIN results, what should the *overall* case status be?
What to build	A function that takes a list of per-rule Results and returns one of: compliant / potential_non_compliance / manual_review_required.
Expected output	A single status string per case, plus 4 passing unit tests.
How to test	Run the unit tests; also manually construct one example matching each of your 3 real demo packages and confirm the aggregated status matches what you'd expect a human to conclude.
Definition of done	All 4 unit tests pass, and the function's output matches your team's own manual judgment on all 3 demo packages.
Common failure	A single UNCERTAIN result on a minor field pushes an otherwise-clean case into manual_review_required, making the demo look overly cautious.
Fallback if it still fails	This is a tuning problem, not a bug — discuss with Member 6 whether that specific field's confidence floor (Part 10) needs adjusting, rather than changing the aggregation logic itself.

♦ **COPY-PASTE PROMPT — Day 3 — Full `evaluate_case()` Aggregation**

Write a Python function `aggregate_case_status(results: list[Result]) -> str`` that takes a list of rule results, each with a status of PASS, FAIL, or UNCERTAIN, and a severity of MAJOR or MINOR, and returns one overall case status: 'compliant' only if all results are PASS; 'manual_review_required' if any result is UNCERTAIN and none are a MAJOR FAIL; 'potential_non_compliance' if any MAJOR-severity result is FAIL, regardless of the others. Write 4 unit tests covering each branch, including a mixed case with both an UNCERTAIN and a MINOR FAIL.

Member 5 — Backend / Database Engineer

Mission: FastAPI endpoints, the SQLite schema, and PDF report generation. Works in lockstep with Member 1 (integration) and Member 4 (rule engine).

Day	Morning	Afternoon	Evening
1	FastAPI official tutorial (Part 14); scaffold backend project	Build POST /scan (upload + save file) endpoint	Design + create the SQLite schema (Part 10 links here)
2	Build POST /extract wired to Member 3's OCR function	Store raw OCR results as JSON in the database	Test /extract end-to-end with a real uploaded photo
3	Build POST /evaluate wired to Member 4's rule engine	Store rule results in the database against the right inspection record	Test the full scan→extract→evaluate chain with Member 1

Day	Morning	Afternoon	Evening
4	Build PDF report generation with ReportLab	Build GET /history and GET /inspection/{id} endpoints	Test PDF output visually on all 3 demo cases
5	Run the Part 26 security pass (file validation, size limits, env vars)	Fix bugs found in final QA	Support deployment/backup demo setup with Member 1

Detailed breakout — Day 1: SQLite Schema + First Endpoint

Objective	A working POST /scan endpoint that saves an uploaded image and creates a database row for it.
What to learn first	A database table is just a spreadsheet the code can query reliably — SQLite stores it in a single file, no server needed.
What to build	A CREATE TABLE statement for `inspections`, plus a FastAPI endpoint that inserts a row per upload.
Expected output	Calling POST /scan with an image returns {"id": 1} (or similar), and a new .db file appears with one row in it.
How to test	Use a free SQLite browser tool (or `sqlite3 labelguard.db "SELECT * FROM inspections;"` in the terminal) to confirm the row was actually inserted.
Definition of done	Three consecutive uploads create three distinct rows with unique, non-overwriting image file paths.
Common failure	Second upload overwrites the first image file because both got the same filename.
Fallback if it still fails	Generate a unique filename using a UUID or timestamp instead of the original filename, and re-verify with two uploads of the same original file.

♦ COPY-PASTE PROMPT — Day 1 — SQLite Schema + First Endpoint

Design a SQLite schema for a table called `inspections` with columns: id (primary key), created_at (timestamp), image_path (text), extracted_fields (text, will hold JSON), rule_results (text, will hold JSON), overall_status (text), officer_note (text, nullable). Give me the CREATE TABLE SQL, a Python function using the built-in sqlite3 module to initialize the database if it doesn't exist, and a FastAPI POST /scan endpoint that accepts an uploaded image file, saves it to disk with a unique filename, and inserts a new row with just the image_path filled in (other fields null for now), returning the new row's id.

Detailed breakout — Day 4: PDF Report Generation

Objective	One function that takes a completed inspection record and produces a real, openable PDF.
What to learn first	ReportLab builds a PDF by adding 'flowable' elements (paragraphs, tables, images) to a document in order, top to bottom.

What to build	A function <code>generate_report_pdf(inspection) -> file_path`</code> producing a one-page report.
Expected output	A real .pdf file that opens correctly and shows all the report content laid out readably.
How to test	Generate a report for each of the 3 demo cases and open all three PDFs to visually confirm nothing overlaps or is cut off.
Definition of done	Report generates without error for all 3 demo cases and is legible when printed/viewed at 100% zoom.
Common failure	Image doesn't appear, or a None value in a field crashes the PDF generation.
Fallback if it still fails	Add a default placeholder ('Not detected') for any missing field before it reaches the PDF function, and confirm the image path exists on disk before passing it to ReportLab's Image flowable.

♦ COPY-PASTE PROMPT — Day 4 — PDF Report Generation

Using ReportLab, write a Python function that takes an inspection dict (fields: image_path, extracted_fields dict, rule_results list, overall_status, created_at) and generates a one-page PDF report with: a title 'LabelGuard Inspection Report', the case date, the product image (scaled to fit), a table of extracted fields and their values, and a list of findings showing rule description, status, severity, and confidence. Save it to a given output path and return that path. Include the exact pip install command and a way for me to open and visually check the generated PDF from the command line.

Member 6 — Research / QA / Product / Pitch Lead

Mission: keep the whole team honest about the law (Part 9), build the demo dataset, own testing, and turn the finished product into a pitch a judge will remember. This role is the difference between a working demo and a winning one.

Day	Morning	Afternoon	Evening
1	Deep-read the Rules + official FAQ (Part 9)	Start building the demo dataset: draft 1 compliant mock label	Draft 1 non-compliant and 1 ambiguous mock label
2	Finish the demo dataset (Part 23); define ground truth per image	Write the first 10 test cases (Part 22)	Start competitor research (Part 29) with live web search
3	Finish competitor research + comparison table	QA-test the rule engine against the dataset with Member 4	Refine ground truth labels based on QA findings
4	Full end-to-end QA of the whole pipeline (all 20+ test cases)	Start drafting the pitch deck (Part 31)	Draft the 3-minute pitch script (Part 32)
5	Finalize pitch deck + speaker notes	Rehearse judge Q&A (Part 34) as a team	Record the offline demo fallback video (Part 38); final team walkthrough

Detailed breakout — Day 1: Reading the Rules Correctly

Objective	Produce the authoritative field list the whole team builds against — get this wrong and every downstream part inherits the error.
What to learn first	The difference between what a regulation requires in principle and what a photo can actually prove — this is the single most important judgment call on the team.
What to build	A filled-in version of the Part 9 rule table: for each declaration, the exact source, whether it's machine-checkable, and why.
Expected output	A short, accurate writeup per declaration that Member 4 can turn directly into a rule JSON object.
How to test	Cross-check every claim against the actual official source PDF/page from Part 9 — not against the LLM's memory alone.
Definition of done	Every declaration in the MVP has a source citation with a rule number and a clear machine-checkable/partial/manual verdict, verified against the primary source, not just the LLM's summary.
Common failure	The LLM confidently states a requirement that isn't actually in the current text, or misses a 2023 amendment.
Fallback if it still fails	Treat any LLM legal summary as a first draft only — always open the actual official PDF/page and confirm the specific clause before it goes in the handbook or the rule engine.

♦ COPY-PASTE PROMPT — Day 1 — Reading the Rules Correctly

I'm going to paste the text of specific rules from the Legal Metrology (Packaged Commodities) Rules, 2011. For each one, tell me in plain language: (1) what it actually requires, (2) whether that requirement could realistically be checked from a single photo of a label using OCR, and (3) if only partially checkable, exactly what part is checkable versus what would still need a human. Do not simplify the requirement into something stricter or looser than what's written. If you are not certain of the current wording, say so explicitly instead of guessing.

Detailed breakout — Day 2: Building the Demo Dataset

Objective	3–10 self-created mock package label photos covering compliant, non-compliant, and ambiguous cases — the entire demo depends on these being good.
What to learn first	'Ground truth' means: before you test the system, you (a human) decide and write down what the correct answer should be for each test image.
What to build	A small folder of labeled mock product photos plus a spreadsheet of expected results per image.
Expected output	3 clear layout specs you can turn into real mock labels within an hour or two.
How to test	Have a teammate who didn't design the labels look at each one and independently guess the

	intended verdict — if they disagree with your ground truth, the label is ambiguous in the wrong way.
Definition of done	3 mock labels exist as real photographable images, each with a written expected verdict, and a teammate's independent read matches that verdict for the compliant and non-compliant cases.
Common failure	The 'non-compliant' mock label is so obviously broken that it doesn't actually test OCR/rule-engine logic, or the 'ambiguous' one is just illegible junk.
Fallback if it still fails	Make violations subtle but real (e.g. a plausible-looking but incomplete address) rather than cartoonishly wrong — subtle violations are what actually demonstrate the rule engine is doing real work.

♦ **COPY-PASTE PROMPT — Day 2 — Building the Demo Dataset**

Help me design 3 mock product label layouts I can create myself (e.g. printed and photographed, or designed as simple images) for testing a Legal Metrology compliance checker: (1) a fully compliant label with all required declarations clearly printed, (2) a non-compliant label missing the manufacturer address and using an ambiguous net-quantity unit, (3) an ambiguous/low-quality case (glare or tiny font) where the correct system behavior is 'manual review required', not a false pass or fail. For each, list exactly what text and layout to include so I can build it in a simple design tool.

Part 19 — Five-Day Master Schedule

The cross-team view. Use this to see how all six tracks converge each day. Full member detail is in Part 18.

Day 1 — Foundation

Deliverables: GitHub repo live · frontend + backend hello-world connected · SQLite schema created · image upload working · first OCR proof-of-concept script running.

Day 2 — Intelligence

Deliverables: OCR wrapped into a reusable, API-exposed function · field extraction producing structured declarations · first 3 rules with confidence scoring.

Day 3 — Compliance

Deliverables: Full rule engine across all 7 declaration categories · violation detection with severity · aggregated compliance result wired end-to-end.

Day 4 — Evidence + Reporting

Deliverables: Evidence highlighting on the image · inspection history persisted · PDF report generation · complete workflow runnable start to finish.

Day 5 — Freeze + Demo

Deliverables: Full regression testing · bug fixing only, no new features · pitch + Q&A rehearsed · offline backup demo recorded · final build tagged in Git.

Gantt-style cross-member view

Member	Day 1	Day 2	Day 3	Day 4	Day 5
1 — Tech Lead	Repo + skeleton integration	API contract + OCR wiring	Rule engine wiring	Evidence + PDF integration	Freeze + bug triage
2 — Frontend	Upload screen	Quality + processing screens	Extracted data + result screens	Evidence + history screens	States polish + rehearsal
3 — OCR/CV	First OCR script	Quality gate + OCR wrapper	Preprocessing + box-drawing start	Evidence highlighting done	Lighting/device test
4 — Rule Engine	Field list + schema	First 3 rules + regex	All 7 categories + aggregation	Threshold tuning + explanations	Regression test + freeze
5 — Backend/DB	Schema + /scan endpoint	/extract endpoint	/evaluate endpoint	PDF + history endpoints	Security pass + bug fixes
6 — Research/QA/Pitch	Legal read + dataset start	Dataset + test cases + competitors	QA pass + ground truth refine	Full QA + pitch draft	Pitch + Q&A + backup video

Part 20 — Daily War Room

Three short meetings a day, every day. Keep them brutally short — the goal is surfacing blockers fast, not status theater.

09:00 — 15-minute Standup

- What did you finish since last night?
- What are you doing today?
- What's blocking you right now?
- Do you need anyone else's output before you can start today's task?

15:00 — 10-minute Blocker Meeting

- Who is currently stuck? (Say it even if you think you'll solve it in 5 minutes — say it anyway.)
- Has this exact blocker already been solved in Part 17's debugging playbook?
- Does solving this need a second person, or just 15 more focused minutes?
- If still stuck at 15:15, escalate to Member 1 immediately — don't wait for the evening review.

21:00 — 20-minute Review

- Demo what you built today to the rest of the team, live, for real — not a description of it.
- Does today's Part 19 deliverable actually exist and work?
- What moved to tomorrow, and does that put the master schedule at risk?
- Update KNOWN_ISSUES.md and TODO.md (Part 41) before leaving for the day — never rely on memory across a night's sleep.

Part 21 — GitHub Workflow

Branch strategy

- `main` — always demoable. Never commit directly to main after Day 1.
- One feature branch per person per task, named `member-N/short-task-name` (e.g. `member-4/net-quantity-rule`).
- Merge into main via Pull Request, reviewed by Member 1 or one other teammate, at least once a day.

Commit naming convention

`type: short summary` — types: feat (new feature), fix (bug fix), refactor, docs, test, chore. Example: `feat: add net quantity regex rule with unit tests`.

Exact commands

```
# Clone the repo (once, per machine)
git clone https://github.com/YOUR-ORG/labelguard.git
cd labelguard

# Start a new task
git checkout main
git pull origin main
git checkout -b member-4/net-quantity-rule

# Save your work
git add .
git commit -m "feat: add net quantity regex rule with unit tests"
git push origin member-4/net-quantity-rule

# Then open a Pull Request on GitHub's website, get it reviewed, and merge.

# After your PR is merged, update your local main
git checkout main
git pull origin main

# Resolving a merge conflict
git pull origin main          # conflict appears here
# open the conflicted file(s), look for <<<<<<, =====, >>>>>> markers
# manually edit to the correct final content, then:
git add <the-file>
git commit -m "fix: resolve merge conflict in rules.py"
git push
```

Emergency recovery commands

Situation	Command
I want to undo my last commit but keep the changes	<code>`git reset --soft HEAD~1`</code>
I want to throw away all my uncommitted changes	<code>`git checkout -- .`</code>
I committed to the wrong branch	<code>`git log`</code> to find the commit hash, <code>`git checkout correct-branch`</code> , <code>`git cherry-pick <hash>`</code>
My branch is badly broken and I want a clean copy of main	<code>`git fetch origin`</code> , <code>`git reset --hard origin/main`</code> (destroys local changes — be sure first)
I accidentally pushed a secret/API key	Rotate the key immediately (treat it as compromised), then remove it from history with the team's help — don't just delete the line in a new commit, the secret is still in history

Part 22 — Testing Strategy

20+ test cases across the categories that actually matter for a photo-based compliance tool. Run these against the demo dataset (Part 23), not production data that doesn't exist yet.

#	Category	Test case	Expected behavior
1	Happy path	Clean, well-lit, compliant label photo	overall_status = compliant, all rules PASS
2	Happy path	Clean, well-lit, non-compliant label (missing MRP)	MRP rule = FAIL, severity Major, overall_status = potential_non_compliance
3	Missing fields	Label photo cropped so consumer care info is out of frame	consumer_care rule = UNCERTAIN, not FAIL
4	Missing fields	Label with genuinely no manufacturer address printed	manufacturer rule = FAIL (this is a real, provable absence, not an OCR failure)
5	Bad OCR	Extremely blurry photo	Image quality check rejects it before OCR runs
6	Blurry image	Mildly blurry but technically readable photo	Quality check passes; OCR confidence may be lower, driving relevant fields to UNCERTAIN
7	Glare	Photo with a bright reflection over the price	Price field UNCERTAIN, not falsely PASS or FAIL
8	Curved package	Photo of a label on a cylindrical bottle	OCR partial read; system should degrade to UNCERTAIN on distorted fields, not crash
9	Multilingual	Label with English + Hindi text mixed	English fields still extracted correctly; Hindi-only fields marked UNCERTAIN if multilingual OCR is out of MVP scope
10	Wrong values	Net quantity present but implausible (e.g. '5g' on a 1kg-looking product)	System flags format-plausible but cannot verify against real fill weight — correct behavior is PASS on format, with a documented limitation, not a false accuracy claim
11	Missing declarations	Country of origin declaration missing on a package that isn't imported	Rule correctly does not apply / is skipped, not flagged as FAIL
12	Low confidence	OCR confidence just below the MEDIUM threshold	overall_status = manual_review_required

#	Category	Test case	Expected behavior
13	API failure	Backend OCR service throws an exception mid-request	Endpoint returns a graceful partial result, not a raw 500 crash visible to the user
14	Database failure	SQLite file locked or missing on save	Upload/inspection still completes in-memory for the current session; error is logged, not silently swallowed
15	Network failure	Frontend loses connection mid-upload	Clear error state shown with a retry option, no infinite spinner
16	Duplicate upload	Same image uploaded twice	Two independent inspection records are created (expected — no dedup required for MVP)
17	Wrong file type	A PDF or .docx uploaded instead of an image	Rejected at upload with a clear error message, not a cryptic crash
18	Oversized file	A 50MB image uploaded	Rejected with a clear size-limit error
19	Empty image	A blank white image uploaded	Quality check or OCR returns 'no text detected'; overall_status = manual_review_required
20	Report generation	Generate a PDF for a case with at least one UNCERTAIN and one FAIL finding	PDF renders both correctly, no crash on mixed statuses
21	History	View inspection history after 5+ saved cases	All 5 appear, most recent first, each opens correctly
22	Evidence alignment	View evidence for a finding on a resized image	Bounding box aligns with the correct region, not offset

QA spreadsheet structure

Test ID	Description	Expected result	Actual result	Pass/Fail	Notes
TC-01	...	...	...	...	...
NOTE Keep this as a real, live spreadsheet (Google Sheets/Excel) owned by Member 6, updated daily from Day 2 onward — not a one-time Day 5 exercise.					

Part 23 — Dataset Strategy

Build a small, legal, ethical, self-created dataset. Do not scrape real brand packaging without a clear right to use it, and never claim government-grade accuracy from a handful of self-made images.

Sources, in order of preference

34. Self-created mock labels — design simple labels yourselves (any design tool), print them, photograph them under a few different lighting conditions. Full control over ground truth and legality.
35. Publicly accessible examples where clearly permitted (e.g. your own household products, photographed by you, used only for internal testing, never redistributed as if official).
36. Controlled test labels deliberately created to be non-compliant or ambiguous for negative-case testing.

Three categories

COMPLIANT	All required declarations present, legible, correctly formatted — expected system output: compliant.
NON-COMPLIANT	At least one required declaration is genuinely and clearly absent or malformed — expected system output: potential_non_compliance.
AMBIGUOUS	Deliberately hard to read (glare, tiny font, partial crop) — expected system output: manual_review_required, NOT a confident pass or fail.

NOTE — On ground truth

Ground truth means a human on the team decides and writes down, before running the system, what the correct answer for each test image should be. Without this written down first, “testing” just becomes tuning the system until it agrees with whatever you already believed — which proves nothing to a judge.

Part 24 — Performance Metrics

WARNING — NEVER INVENT METRICS

If the team only has a small, self-built prototype dataset, every number reported must be labeled “prototype evaluation” — never presented as if it were a validated, government-grade accuracy figure.

Metric	Formula	What it tells you
OCR field accuracy	(fields correctly read) / (total fields present)	How often OCR gets the right text for a field that's actually there
Precision	True Positives / (True Positives + False Positives)	Of the violations you flagged, how many were real
Recall	True Positives / (True Positives + False Negatives)	Of the real violations, how many you actually caught
F1 score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	A single balanced score combining precision and recall
False positive rate	False Positives / (False Positives + True Negatives)	How often a compliant field gets wrongly flagged
False negative rate	False Negatives / (False Negatives + True Positives)	How often a real violation is missed
Processing time	End-to-end wall-clock time from upload to result	Whether the system is fast enough to be practically useful
Confidence calibration (if feasible)	Compare stated confidence vs actual correctness rate at that confidence level	Whether “90% confidence” findings are actually right about 90% of the time — only attempt this if the dataset is large enough to be meaningful; otherwise state it as future work

Part 25 — Threat / Failure / Loophole Analysis

Pressure-test the idea like a hostile SIH judge. 30 attacks, with a weak answer to avoid and a strong answer to give.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
How do you know the law?	"We read it online."	"Every rule we implemented cites the exact clause in the Legal Metrology (Packaged Commodities) Rules, 2011, listed in our source-of-truth appendix, verified against the official gazette text."	Show Part 9's source table live.
What if OCR is wrong?	"It's usually right."	"That's exactly why every finding carries a confidence score, and anything below our threshold is routed to manual review instead of an automatic verdict — see our human-in-the-loop design."	Show the Part 8 confidence tiers and a live UNCERTAIN case.
What if the package is curved?	"We didn't test that."	"Curved packaging distorts text, which lowers OCR confidence on that region — our system correctly degrades to manual review rather than guessing, and we've documented it as a known limitation, not hidden it."	Part 36 limitations list.
What if text is tiny?	"Should still work."	"Below a resolution/legibility threshold our quality gate asks for a retake before OCR even runs, which is more honest than returning a low-quality guess."	Live demo of the quality-gate rejection.
What if text is multilingual?	"We support all languages."	"Our MVP is English-first by design choice for the 5-day scope; multilingual OCR is a documented should-have, not a false claim of full coverage today."	Part 4 MVP scope table.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
What if the label is reflective/glare?	"We'll fix it later."	"Glare is one of the documented failure modes that routes a finding to manual review — exactly the behavior you'd want from a legally cautious tool."	Part 36.
What if the AI hallucinates?	"Our AI doesn't hallucinate."	"The compliance decision is never made by a generative AI model — it's made by a deterministic, versioned rule engine. AI is only used for assistive steps like OCR cleanup, which are independently checkable against the image."	Part 12 architecture diagram.
What if the rules change?	"We'll update the code."	"Rules are stored as versioned JSON, separate from application code, specifically so a legal update doesn't require a software re-architecture — just an edited/added rule file."	Part 10 rule schema.
What if legal interpretation changes/is disputed?	"The law is the law."	"We explicitly mark disputed or judgment-dependent requirements as 'manual verification required' rather than encoding a single interpretation as fact."	Part 9 category legend.
Can this replace an inspector?	"Yes, eventually."	"No — by design. It's inspection assistance that makes one inspector faster and more consistent; every consequential decision still requires human sign-off."	Part 9 non-negotiable language rules.
Can it work offline?	"No."	"The core OCR + rule engine can run fully offline on a local machine; only should-have features like cloud OCR APIs would need	Architecture diagram + Part 38.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
		connectivity, and we have a fallback for that (Part 38)."	
What if someone manipulates the image?	"We don't handle that."	"That's a real limitation we disclose openly — image-tampering detection is future-roadmap work, not claimed in the prototype."	Part 36/37.
Can GPS be trusted?	"We don't use GPS."	"GPS/location isn't part of this MVP's compliance logic at all — we don't make claims we haven't built."	Part 4 scope.
How do you validate?	"It works on our test images."	"We built a labeled dataset with human-decided ground truth across compliant/non-compliant/ambiguous cases and measured precision/recall/F1 against it — explicitly labeled as prototype-scale evaluation, not a production claim."	Part 23/24.
What is your accuracy?	A specific invented percentage.	"On our prototype dataset we measured [actual measured numbers] — we're not going to claim a number we haven't tested, and any number we give is prototype-scale, not validated at production scale."	Real numbers from Part 24, run on the real dataset before the pitch.
Why not just use Google Lens?	"Google Lens is bad."	"General-purpose OCR tools read text but don't know India's Legal Metrology declaration requirements, don't produce a structured compliance verdict, and don't generate evidence-linked inspection reports — that domain logic is the actual product."	Part 29 competitor table.
Why not just use OCR?	"OCR isn't enough." (true	"OCR is step 3 of an 11-step	Part 5 workflow diagram.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
	but underdeveloped)	pipeline — the value is in structured extraction, the deterministic rule engine, evidence mapping, and human-review routing built on top of it.”	
Why is this better than a chatbot?	“Chatbots are generic.”	“A chatbot gives an unfalsifiable text answer. We give a structured, evidence-linked, rule-traceable verdict a supervisor can audit without re-doing the work.”	Part 7 evidence-first explanation.
Why is this AI at all?	“AI is trendy.”	“Computer vision and OCR are the only practical way to convert a photograph into structured text at the speed a field inspection needs — the AI is doing perception, not judgment.”	Part 12.
What is your USP?	“We use AI + OCR + LLM.”	“Evidence-first compliance: every finding is traceable from rule to field to image region to confidence to human review — not a bare AI verdict.”	Part 7.
What is the business model?	“We'll figure it out later.”	“Government licensing to Legal Metrology departments is the primary path, with enterprise/manufacturer self-check as a secondary market — detailed in our business model canvas.”	Part 28.
Who pays?	“Everyone will want this.”	“Primarily the sponsoring department/state Legal Metrology enforcement bodies, via a licensing or on-premise deployment model — the same buyer as the problem statement's	Part 28.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
		sponsor.”	
What happens after SIH?	“We’ll keep building.”	“Our roadmap moves from this prototype to a supervised pilot with a real inspection team, using their feedback to expand rule coverage and validate accuracy at scale before any production claim.”	Part 37.
What's your false positive rate?	“Very low.”	“Measured on our prototype dataset at [X]% — and structurally, our UNCERTAIN tier exists specifically to convert likely false positives into manual review instead of false accusations.”	Part 24 + Part 8.
Is your dataset representative of real Indian retail packaging?	“Yes.”	“No — it's a small, self-created prototype dataset, which we disclose explicitly. Representativeness at scale is exactly what a pilot phase (Part 37) would need to establish.”	Part 23/36.
How do you handle a manufacturer disputing a flagged violation?	“The system is always right.”	“The system never issues a final verdict — every flagged case requires human review before any formal action, and the evidence image is attached specifically so a dispute can be resolved by looking at the actual photo.”	Part 7/8.
What stops an inspector from just ignoring the tool?	“They won't.”	“Adoption risk is real or any new tool; that's why the workflow is designed to make the inspector faster, not to add a new required step — it should feel like it saves them time from day one.”	Part 3 user journey.

Judge question	Weak answer (avoid)	Strong answer (use)	Evidence to show
Could this be biased against certain manufacturers/products?	"AI isn't biased."	"The rule engine applies identical, explicit rules to every image regardless of brand; the main bias risk is OCR performing worse on some fonts/languages, which is exactly why we route low-confidence cases to a human rather than auto-deciding."	Part 8/36.
What's stopping someone from gaming the photo (best angle, best lighting) to pass?	"They can't."	"A prototype photo-based tool can be gamed by a motivated bad actor; that's a known limitation of any photo-based inspection method (including a human inspector's eyes) and is disclosed as future work around tamper/liveness detection."	Part 36/37.
Why should the government trust a student prototype with legal consequences?	"It's very accurate."	"They shouldn't trust it with legal consequences yet — that's exactly why the architecture keeps a human as the final decision-maker at every consequential step, by design, not as an afterthought."	Part 8/9 non-negotiable language rules.

Part 26 — Security

Appropriate for a 5-day prototype — not enterprise-grade, but not careless either.

- File upload validation: check both file extension AND actual file content/MIME type, not just the extension string.
- Size limits: reject uploads above a sane threshold (e.g. 10MB) before processing.
- File type validation: only accept image formats the pipeline actually supports (jpg, png).
- Path traversal protection: never build a file save path directly from user-supplied filenames — generate your own safe filename (UUID) instead.
- API key protection: any third-party API key lives in an environment variable / `.env` file, never hardcoded or committed to Git (add `.env` to `.gitignore` on Day 1).
- Environment variables: use them for anything that differs between a teammate's laptop and the demo machine.
- Input sanitization: never directly interpolate user input into a raw SQL query — use parameterized queries.
- Rate limiting: optional for a 5-day demo, skip unless there's time left over.
- Privacy: see Part 27.
- Image deletion/retention: for the demo, a documented, simple retention policy (e.g. “images kept for the duration of the demo session”) is enough — don't over-engineer this.
- Database safety: back up the SQLite file before any risky schema change, especially close to the demo.

Part 27 — Privacy / Ethics

- Package images: only photograph packaging, not people, bystanders, or unrelated background details where avoidable.
- Inspector data: the MVP doesn't need real inspector personal data — a single shared demo login is enough (Part 6).
- User accounts: don't build a multi-user auth system just to “look complete” — it adds real security surface area for no MVP benefit (Part 4 DO NOT BUILD list).
- Logs: don't log more than needed to debug; avoid logging full image bytes into text logs.
- Retention: keep only what's needed to run the demo and any post-hackathon writeup; don't accumulate images indefinitely without a reason.
- Data minimization: the system should only ask for what it needs to check compliance — a product photo — not unrelated personal or business data.

Part 28 — Business Model

Realistic, not fantasy. No invented revenue projections.

Customer	Problem	Value LabelGuard offers
Legal Metrology departments (state/central)	Manual, slow, inconsistent field inspection	Faster per-inspector throughput + documented, evidence-backed case files
Inspection/enforcement agencies	Same, at larger scale across many inspectors	Standardized checklist + auditable history across a whole team
Large manufacturers (compliance/QA teams)	Risk of a labeling mistake reaching the market	Optional self-check before print, reusing the same evidence-first logic
Testing/inspection organizations	Need a documented, repeatable inspection method	A structured report format usable as an audit artifact

Pricing/procurement possibilities (pros/cons, not a fixed decision)

Model	Pros	Cons
Government licensing (departmental)	Matches the sponsor's own likely procurement path; clear single buyer	Long government procurement cycles
Enterprise SaaS (manufacturer self-check)	Faster sales cycle than government	Different buyer/positioning than the original PS; would need separate validation
On-premise deployment	Appeals to a department wanting data control	Higher deployment/support overhead for a small team
API licensing	Lets other systems integrate compliance checks	Requires production-grade reliability/SLA the prototype doesn't have yet

Business Model Canvas

Key Partners: Legal Metrology departments, IILM (training body), OCR/CV open-source ecosystem	Key Activities: rule maintenance, OCR pipeline upkeep, evidence UI, report generation	Value Proposition: evidence-first, human-in-loop compliance pre-screening that speeds up inspection
Key Resources: rule library, demo/eval dataset, small engineering team	Customer Relationships: departmental pilot → supported rollout	Channels: direct government engagement, SIH/departmental introduction
Customer Segments: Legal Metrology inspectors/departments; secondary: manufacturers	Cost Structure: engineering + hosting + rule-maintenance time	Revenue Streams: government licensing, optional enterprise SaaS/API licensing

NOTE

Do not present invented revenue figures, user counts, or market-size numbers in the pitch unless they come from a cited real source. “We estimate significant time savings per inspector based on our workflow comparison” is honest; a specific fabricated ₹-crore market-size slide is not.

Part 29 — Competitor Analysis

Researched via live web search. No direct, verified competitor was found that (a) targets India's Legal Metrology (Packaged Commodities) Rules specifically and (b) is built for a field inspector photographing already-shelved retail packaging. The tools below are real, currently-operating products in adjacent spaces.

Solution	What it actually does	Category	LabelGuard's difference
GlobalVision (globalvision.co)	AI-assisted label/artwork proofreading for CPG brands, checking claims/nutrition/spec-matching before print	Adjacent solution	Built for brand-owner pre-print QC, not government field inspection of already-printed retail packages; not India-Legal-Metrology-specific
Artwork Flow (artworkflowhq.com)	Label/artwork compliance workflow software with customizable rule checks	Adjacent solution	A general artwork-review workflow tool, not scoped to India's Rules or evidence-linked field inspection
iFactory AI Label Verification (ifactoryapp.com)	FMCG line inspection: allergen/barcode/nutrition verification with an audit trail	Adjacent solution	Factory production-line QC (per-unit, high-speed), not a photo-based field-inspector tool
SwitchOn DeepInspect (switchon.io)	Machine-vision label quality/defect inspection on production lines	Adjacent solution / generic tool	Industrial line-camera hardware+software, not a lightweight photo-upload inspector app
Jidoka (jidoka-tech.ai)	OCR + template matching for label placement/print-quality checks on packaging lines	Adjacent solution	Same production-line framing as above, not field/legal-inspection framing
GetGenAI (getgen.ai)	AI review of packaging/label text against regulatory claims across markets	Adjacent solution	Multi-market brand-compliance copywriting checks, not India-specific declaration verification with evidence mapping
Generic OCR apps / Google Lens-style tools	Read text from an image	Generic tool	No domain rule engine, no compliance verdict, no evidence mapping, no inspection report
Generic document-AI platforms	Extract structured fields from documents generally	Generic tool	Not tuned to packaged-commodity declarations or Legal Metrology rule structure

NOTE

If a judge names a specific product not on this list, the honest answer is: "We researched the space and didn't

Solution	What it actually does	Category	LabelGuard's difference
find a direct India-Legal-Metrology field-inspection competitor; happy to look that one up.” Never claim exhaustive competitor knowledge.			

Part 30 — Competitive Moat

Potential moat	Realistic now (prototype)	Realistic later (post-pilot)
Domain-specific rule library tied to actual Indian Rules	Yes — core of the MVP	Deepens with every amendment tracked over time
Verified compliance rules with source citations	Yes, at MVP scope	Expands to full Rules coverage + state-level variations
Inspection dataset	No — too small to be a moat yet	Grows into a real asset only after a genuine pilot with real inspectors
Evidence mapping UX	Yes — already a differentiator vs generic tools	Refined with real user feedback
Feedback loops (officer overrides improving the system)	No — not in 5-day scope	A real moat once a pilot generates override data
Government workflow integration	No	Strong moat if achieved — hard for a new entrant to replicate quickly
Multilingual capability	No — should-have/optional	Meaningful differentiator for pan-India deployment
Offline inspection capability	Partial (core pipeline can run offline)	Valuable for rural/low-connectivity deployment
Audit trails	Yes, at basic level (history + evidence)	Strengthens into a compliance-grade audit system

Part 31 — Pitch Deck (10 Slides)

Slide 1 — Title

Exact text	LabelGuard — “Scan the Label. Find the Violation. Show the Evidence.” SIH26034 — Ministry of Consumer Affairs, Food & Public Distribution
Visual recommendation	Clean product logo/wordmark, a single sharp product-label photo in the background, muted color.
Diagram	None.
Speaker notes	State the team name, the problem statement ID, and the one-line tagline. Keep it under 10 seconds.
What NOT to put	Don't list all 6 team members' full bios here — save for a closing/thank-you slide if one exists.

Slide 2 — Problem

Exact text	Manual label inspection under the Legal Metrology (Packaged Commodities) Rules, 2011 is slow, inconsistent, and hard to document — one inspector, one package, one line of fine print at a time.
Visual recommendation	A photo of a busy shelf of packaged products, or a simple icon of a magnifying glass over tiny text.
Diagram	None, or a simple “today's workflow” icon row (see Slide 3).
Speaker notes	Ground it in a concrete image: an inspector checking a shelf full of products, reading fine print by eye.
What NOT to put	Don't overstate the crisis with invented statistics (e.g. a fabricated national violation rate).

Slide 3 — Current Workflow

Exact text	Today: Inspector visits → reads label manually → checks memory of requirements → writes up findings by hand → slow, inconsistent, hard to audit later.
Visual recommendation	A simple 4-icon horizontal flow (visit → read → recall rules → handwritten notes).
Diagram	Simple linear flow diagram, 4 boxes.
Speaker notes	This slide exists purely to set up the contrast with Slide 4 — keep it to 15 seconds.
What NOT to put	Don't editorialize about inspectors being bad at their jobs — the problem is workflow tooling, not people.

Slide 4 — Solution

Exact text	LabelGuard: photograph the package → automatic OCR + extraction → checked against an explicit rule engine → evidence-linked result → human review when uncertain → instant report.
------------	--

Visual recommendation	The same 4-box flow from Slide 3, now compressed into one fast LabelGuard icon with a clock showing dramatically less time.
Diagram	Before/after comparison diagram.
Speaker notes	This is the “aha” slide — say the one-sentence pitch from Part 1.11 word for word here.
What NOT to put	Don't say “we use AI to solve compliance” without the follow-through of Slide 7's evidence-first explanation.

Slide 5 — How It Works

Exact text	Upload/Camera → Quality Check → OCR → Extraction → Rule Engine → Evidence → Human Review → Report → History.
Visual recommendation	The full Part 5 pipeline diagram, simplified to fit one slide (group into 4-5 visual stages, not all 11 steps individually).
Diagram	Horizontal pipeline diagram with icons.
Speaker notes	Move fast here — this is context for the live demo coming right after, not the main event.
What NOT to put	Don't read every pipeline step aloud word for word; let the diagram do the work.

Slide 6 — Technology

Exact text	Next.js + FastAPI + PaddleOCR + OpenCV + a deterministic Python/JSON rule engine + ReportLab. Built in 5 days by a 6-person team.
Visual recommendation	Simple stacked logos/labels of the 5-6 core technologies.
Diagram	Optional: the Part 12 architecture diagram (AI's role vs the rule engine's role).
Speaker notes	Emphasize the rule engine is NOT an LLM — this single sentence pre-empts several Part 25/34 questions.
What NOT to put	Don't list every library used — only the ones a judge would recognize or care about.

Slide 7 — USP / Evidence-First Compliance

Exact text	Every finding: Rule → Field → Detected Value → Violation → Image Region → Confidence → Human Review. Never a bare ‘AI says non-compliant.’
Visual recommendation	The Part 7 example finding card, rendered exactly as it appears in the product.
Diagram	The rule→field→...→human-review chain as a vertical flow.
Speaker notes	This is the single most important slide in the deck — slow down here, even if rushing elsewhere.
What NOT to put	Don't bury this USP under generic AI buzzwords.

Slide 8 — Validation / Results

Exact text	Tested against a [N]-image prototype dataset across compliant / non-compliant / ambiguous cases. Precision [X], Recall [Y], F1 [Z] — prototype-scale evaluation.
Visual recommendation	A simple bar chart of the real measured metrics from Part 24, run on the real dataset.
Diagram	Bar chart, not a table, for a pitch deck — easier to read from a distance.
Speaker notes	State plainly that this is prototype-scale, not production-validated — credibility over spin.
What NOT to put	Never put a fabricated or unmeasured number here, even a plausible-sounding one.

Slide 9 — Impact / Business / Scalability

Exact text	Primary path: departmental licensing to Legal Metrology enforcement bodies. Scales by adding rule coverage and multilingual OCR, not by re-architecting.
Visual recommendation	The Part 28 Business Model Canvas, condensed to 3-4 boxes.
Diagram	MVP → Pilot → Government Pilot → Production roadmap arrow (from Part 37).
Speaker notes	Keep numbers honest and qualitative unless you have a cited, sourced figure.
What NOT to put	Don't put a fabricated market-size (₹ crore TAM) number without a citation.

Slide 10 — Future Roadmap

Exact text	Next: multilingual OCR, offline mode, expanded rule coverage, a supervised pilot with a real inspection team, and a centralized compliance knowledge base.
Visual recommendation	A simple horizontal roadmap timeline.
Diagram	MVP → Pilot → Government Pilot → Production (same as Part 37).
Speaker notes	End on the honest, credible note: this is a strong prototype with a clear, disclosed path to real deployment — not a finished product.
What NOT to put	Don't promise a specific delivery date for production deployment you don't control.

Part 32 — 3-Minute Pitch (Word-for-Word)

HOOK

Every packaged product on a shelf is legally required to tell you who made it, how much is inside, what it costs, and how to reach them if something's wrong. Right now, the only way to check that is a human reading fine print, one package at a time.

PROBLEM

Legal Metrology inspectors are asked to verify this across thousands of SKUs, manually, from memory, under shop lighting, then write it all up by hand. It's slow, and it's inconsistent between inspectors.

SOLUTION

LabelGuard turns a single photo of a package into a structured compliance check. It reads the label, pulls out the required declarations, and checks them against the actual rules — not a guess, an explicit rule engine built from the Legal Metrology Rules, 2011.

DEMO

Watch: this package is clean — green, done in seconds. This one is missing a manufacturer address — LabelGuard shows exactly where on the image and how confident it is. And this one is a bad photo — glare, hard to read — and LabelGuard is honest about that too: manual review required, not a false guess.

USP

That last part is the whole point. We never let an AI make the legal call. A deterministic rule engine decides; the AI only helps read the image. Every result comes with evidence and a confidence score, and anything uncertain goes to a human — always.

TECHNOLOGY

Under the hood: OCR and computer vision to read the label, a rule engine written in plain Python against the actual Rules, and a report generator — built in five days by six of us, none of whom had shipped a web app before this week.

IMPACT

This doesn't replace an inspector. It gives one inspector the ability to screen far more products per day, with a documented, evidence-backed record for every case they flag.

CLOSING

We're not claiming this is production-ready. We're showing you a working pipeline, honest about its limits, with a clear path from this prototype to a real pilot. That's LabelGuard.

Part 33 — 90-Second Demo Sequence

Three mock packages, exact sequence and dialogue. Rehearse this until it's muscle memory — see Part 38 for the offline fallback if live demo fails.

Time	Package	Action on screen	Exact dialogue
0:00–0:10	1. Compliant	Upload the clean label photo	“First, a normal, fully compliant package. Watch how fast this goes.”
0:10–0:25	1. Compliant	Quality check passes → OCR runs → extraction → result: green	“Quality check passes, OCR reads the label, and— compliant, all declarations present. Seconds, not minutes.”
0:25–0:35	2. Non-compliant	Upload the label missing the manufacturer address	“Now a real-world case — this one's missing something.”
0:35–0:55	2. Non-compliant	Result: potential non-compliance → tap the finding → evidence view with highlighted region + confidence	“LabelGuard flags the manufacturer address as incomplete — 94% confidence — and shows exactly where on the image. This is the evidence-first part: nothing is just asserted.”
0:55–1:10	3. Ambiguous/low-quality	Upload the glare/blurry photo	“And the honest case — a bad photo, the kind you'd actually get in the field.”
1:10–1:20	3. Ambiguous/low-quality	Result: manual review required, not a guess	“Instead of guessing, LabelGuard says: manual review required. That's the human-in-the-loop design working exactly as intended.”
1:20–1:30	Report	Generate + open the PDF report for case 2	“And every case ends with a report the inspector can file immediately. That's LabelGuard.”

Part 34 — Judge Q&A (50+ Questions)

Organized by category for fast lookup during prep. Answers are concise references — expand verbally in the room, don't read them verbatim.

Problem

Why does this problem matter?	Manual, memory-based inspection across huge numbers of SKUs is slow and inconsistent; a documented, evidence-based first pass helps inspectors cover more ground reliably.
Who specifically benefits?	Legal Metrology field inspectors first, then supervisors reviewing flagged cases, then (optionally) manufacturers self-checking labels.
Is this problem already solved?	Not for this specific use case — see Part 29; adjacent commercial tools exist for FMCG pre-print artwork QC, not India-Legal-Metrology field inspection.
How did you validate this is a real problem and not assumed?	From the problem statement itself, issued by the actual regulator (Ministry of Consumer Affairs, Department of Consumer Affairs), plus our own research into the Rules and existing inspection workflow descriptions.

Technical

Walk me through your architecture.	Image → quality gate → OCR → field extraction → normalization → deterministic rule engine → compliance result → evidence mapping → human review when uncertain → report → history (Part 5/12).
Why FastAPI over Node/Django?	Minimal boilerplate, automatic docs, same language (Python) as our OCR/CV/rule-engine code — reduces context switching for a beginner team on a 5-day clock.
Why PaddleOCR over Tesseract?	Stronger out-of-the-box performance on messy real-world layouts and multilingual text, which matches photographed retail labels better than Tesseract's cleaner-document sweet spot (Part 11).
Why SQLite instead of a 'real' database?	It's a single file, zero setup, and entirely sufficient for a single-server 5-day prototype; PostgreSQL would be premature complexity (Part 13).
What was the hardest technical part?	Getting OCR output reliably mapped back to accurate bounding boxes on the original image for evidence highlighting — coordinate/scale mismatches are an easy silent bug (Part 17).
How do you handle a package your rule engine has no rule for?	It's evaluated only against the rules that exist; anything genuinely outside MVP scope is not claimed as checked —

	we're explicit about what's covered (Part 4/9).
--	---

AI

Where exactly does AI/ML appear in your system?	OCR (text recognition) and, optionally, LLM-assisted normalization of messy extracted text — never the compliance decision itself (Part 12).
Did you train a custom model?	No — we deliberately used existing, mature open-source OCR/CV tools rather than training anything, which was the right call for a 5-day timeline (Part 4 DO NOT BUILD list).
Isn't a rule engine 'not really AI'?	Correct, intentionally — for a legal-adjacent decision, deterministic and explainable beats a black box, even a very accurate one (Part 12).
Could you add a fine-tuned model later?	Yes, as a future-roadmap item to improve OCR/extraction accuracy — but the rule engine stays deterministic regardless (Part 37).

Legal

Is this legal advice?	No — explicitly not. Output uses language like 'potential non-compliance' and 'requires human review,' never a legal certification (Part 9).
What's your source for the rules you implemented?	The official Legal Metrology (Packaged Commodities) Rules, 2011 gazette text and the Department of Consumer Affairs' published Rules page, cited per-rule (Part 9).
What if a rule you implemented is actually outdated?	Rules are versioned and dated; a documented version-check date is part of our source table, and updating a rule is a JSON edit, not a re-architecture (Part 10).
Who is legally responsible if the tool is wrong?	The tool never issues a final legal determination — a human reviewer is responsible for any formal action, by explicit design (Part 8/9).

Security

How do you secure uploaded images?	Basic file-type/size validation, safe generated filenames (no path traversal), and environment-variable-based secrets management appropriate to a prototype (Part 26).
What about API keys?	Never hardcoded or committed to Git; stored in environment variables and excluded via .gitignore from Day 1 (Part 26).
Is user data encrypted?	At prototype scale we keep the surface area small (no unnecessary personal data collected, Part 27); production-grade encryption/at-rest security is future-roadmap work (Part

	37).
--	------

Business

What's your business model?	Primary: government/departmental licensing to Legal Metrology enforcement bodies; secondary: enterprise self-check SaaS (Part 28).
What's your revenue projection?	We don't have a fabricated figure — the honest answer at prototype stage is that revenue depends on a real pilot's outcome and departmental procurement terms.
Who is your first customer?	Realistically, the sponsoring department itself or a state Legal Metrology enforcement office, via a pilot engagement (Part 28/37).
What does it cost to run?	At prototype scale, effectively just compute/hosting; a real cost model would need production-scale usage data we don't have yet.

Scalability

Does this scale to millions of inspections?	The architecture (stateless pipeline + swappable database) doesn't structurally block it, but we haven't load-tested at that scale — that's explicitly future work (Part 37).
What would you change for a state-wide rollout?	Move from SQLite to PostgreSQL, add multilingual OCR, and formalize rule versioning/change management across all deployed instances (Part 13/37).
How do you push a rule update to every deployed instance?	Rules are separate JSON files from application code specifically so this can become a config/rule-sync problem, not a redeploy-the-app problem — the sync mechanism itself is future work (Part 10/37).

Validation

How many images did you test on?	[State your team's real number] — a self-built prototype dataset, explicitly not claimed as representative at national scale (Part 23/24).
What's your precision and recall?	[State your team's real measured numbers from Part 24] — prototype evaluation only.
Did anyone outside your team validate this?	At hackathon scope, no independent validation exists yet; that's exactly what a pilot phase would provide (Part 37).

Competition

Who are your competitors?	No direct verified competitor in this exact space; several
---------------------------	--

	adjacent FMCG artwork-QC and label-verification tools exist globally (Part 29).
What stops a big company from copying this?	In the short term, little — the real moat is domain-specific rule depth, evidence UX, and eventually a real inspection dataset from a pilot (Part 30).
Why hasn't the government already built this?	We can't speak to internal government roadmaps; we can say this problem statement being issued for SIH26034 suggests it's an open, current need.

Deployment

How would this actually get deployed in a department?	Most realistically as an on-premise or departmentally-hosted web app, integrated into existing inspection workflows via a pilot (Part 28/37).
Can it run without internet in the field?	The core OCR + rule engine pipeline can run fully offline on a local device; only optional cloud-OCR paths need connectivity (Part 11/38).
What happens if your server goes down during a real inspection?	At prototype scale that's a real limitation; a production version would need redundancy/offline-first design as a roadmap item (Part 37/38).

Feasibility

How long did this actually take to build?	5 days, 6 people, heavy LLM-assisted development — exactly the scope disclosed throughout this handbook.
What didn't you have time to build?	See our Feature Kill List and SHOULD/OPTIONAL tiers (Part 4) — we can list exactly what's out of scope and why.
Could a real department realistically adopt this as-is?	Not as-is — it's a working prototype demonstrating the approach, with an honest, disclosed gap to production (Part 36/37).
What's the single biggest risk to this becoming real?	Validating OCR/rule accuracy at real-world scale across the huge diversity of Indian packaging, fonts, and photography conditions — which is precisely what a pilot phase is for (Part 36/37).

Part 35 — Research Paper Outline

If the team writes a short paper alongside the prototype (common for SIH documentation), use this structure. Guidance on what to actually write is in the right column — don't pad sections with filler.

Section	What to actually write
Abstract	3-4 sentences: problem, approach, what was built, honest scope (prototype, not validated at production scale).
Introduction	The regulatory context and why manual inspection doesn't scale; cite the actual PS and the Rules.
Problem	Expand Part 2's deconstruction — what specifically the PS asks for.
Related Work	Part 29's competitor/adjacent-solution research, honestly categorized.
System Architecture	Part 5/12's pipeline and AI-vs-rule-engine split, with the architecture diagram.
Methodology	How OCR, extraction, and rule evaluation actually work — Parts 10/11.
OCR	Engine chosen, why, preprocessing steps, and measured performance on your dataset.
Rule Engine	Schema, example rules, and the deterministic design rationale (Part 10).
Evidence Mapping	How bounding boxes are linked to findings (Part 7).
Human-in-the-Loop	The 3-tier confidence design and why it exists (Part 8).
Dataset	Exactly how the prototype dataset was built (Part 23) — be explicit about its small size and self-created nature.
Experimental Setup	What images, what hardware, what thresholds were used.
Results	Real measured numbers only (Part 24) — labeled as prototype-scale.
Limitations	Part 36, verbatim honesty — don't soften this section.
Ethics	Part 27's privacy/data-minimization points.
Future Work	Part 37's roadmap.
Conclusion	Restate what was actually demonstrated, not what is aspired to.
References	Part 56 — only verifiable, real sources.

Part 36 — Limitations

NOTE — Say all of this out loud, unprompted, if asked — or even if not

Acknowledging limitations openly is what makes a legal-adjacent AI prototype credible to a technical judge. Hiding them is far more damaging if discovered than stating them up front.

- OCR errors: any OCR engine will misread some text, especially small or stylized fonts.
- Poor lighting: dim or unevenly lit photos reduce extraction reliability.
- Curved packaging: text on bottles/cylindrical containers distorts and is harder to read reliably.
- Tiny text: below a certain size, legibility drops for both OCR and human eyes alike.
- Reflective surfaces: glare on foil or glossy packaging can obscure text entirely.
- Legal ambiguity: some requirements involve judgment a photo alone can't resolve (Part 9's PARTIAL rows).
- Rule updates: the Rules are amended periodically; the prototype's rule set reflects a specific checked date, not a live legal feed.
- Incomplete machine-verifiability: some declarations (e.g. whether an MRP matches the true fill weight) cannot be verified from an image at all.
- Multilingual edge cases: the MVP is English-first; regional-language-only labels are not fully covered.
- Dataset size: the prototype was evaluated on a small, self-created dataset, not a large, independently validated one.

Part 37 — Future Roadmap

MVP → College prototype → Pilot → Government pilot → Production.

- Multilingual OCR coverage for regional languages
- Offline-first mobile deployment for field use with poor connectivity
- Mobile app wrapper for direct in-field camera capture
- Advanced computer vision (perspective correction for curved/angled packaging)
- Rule versioning and change-management tooling as the Rules are amended
- A centralized compliance knowledge base shared across deployments
- Analytics/dashboards for supervisors across many inspectors
- Integration with existing departmental IT/case-management systems
- Enterprise/on-premise deployment options
- Formal audit-trail features for legal defensibility

Part 38 — Disaster Recovery

Failure	Plan
Laptop failure	Repo is on GitHub, not just one laptop; any teammate's machine with the repo cloned and dependencies installed can take over the demo.
Internet failure	Run the full stack locally (frontend + backend + SQLite + local OCR) — none of the MVP's core path requires internet if PaddleOCR/Tesseract are installed locally.
LLM unavailable	Development should already be done by Day 5; an LLM outage on demo day shouldn't block anything that isn't still being actively coded.
OCR API unavailable (if a cloud OCR path was used)	Fall back to the local PaddleOCR/Tesseract pipeline, which should already be the primary path per Part 11.
Deployment failure	Fall back to running everything on the presentation laptop directly — a local demo that works beats a broken cloud URL.
Git corruption	Any teammate's local clone plus GitHub's remote copy is a full backup; worst case, re-clone fresh.
Database corruption	Keep a backup copy of the SQLite file from the last known-good state; worst case, re-seed with the 3 demo cases before presenting.
Presentation laptop failure	A second laptop, pre-configured identically, ready as a hot spare (set this up on Day 5 morning, not at the door).

Offline Demo Mode

NOTE

Build this deliberately on Day 5, don't leave it as an afterthought:

1. Pre-run all 3 demo packages through the full pipeline and save every intermediate result (OCR output, rule results, evidence images, generated PDF) to disk.
2. Build a simple 'replay' mode/script that displays these saved results in the UI without re-running the pipeline live.
3. Also record a clean screen-capture video of one full successful run, as the absolute last resort if even local replay fails.

The demo must still work using preloaded images/results even with zero network access and zero live processing.

Part 39 — “If You Are Stuck” Decision Tree

I cannot run the frontend

- Check Node is installed and is an LTS version (``node -v``)
- Check npm is installed (``npm -v``)
- Check package.json exists and matches the folder you're running ``npm run dev`` from
- Ask the LLM using the Part 17 universal debug prompt with the exact terminal error pasted in
- Try a clean install: delete `node_modules` and `package-lock.json`, then ``npm install`` again
- Fallback: use ``npx create-next-app`` to generate a fresh known-good template and re-apply your changes on top of it

I cannot connect the backend to the frontend

- Check the backend is actually running and note its exact port
- Check for a CORS error specifically in the browser console
- Check the exact endpoint path matches between frontend fetch call and backend route decorator
- Inspect the Network tab in browser dev tools for the real request/response
- Ask the LLM using the Part 17 universal debug prompt with both pieces of code and the Network tab details

OCR is bad on a specific image

- Apply preprocessing (grayscale → contrast → denoise), one step at a time, re-testing after each
- Try cropping to just the label region before running OCR
- Switch to the fallback OCR engine (Tesseract) on the same image and compare
- If a specific demo image is the problem, replace it with a cleaner backup image — don't sink the whole afternoon into one photo

The rule engine gives a wrong result

- Print/inspect the extracted JSON fields going INTO the rule engine — is the input actually correct?
- Inspect the specific rule's regex/logic in isolation (test it in a plain Python shell, not inside the app)
- Inspect the normalization step — did a unit/date/currency get mangled before the rule saw it?
- Write a unit test that reproduces the exact wrong case, then fix until that test passes

PDF report generation is broken

- Print every value going into the PDF function right before use — is something None/missing?
- Confirm the image file path actually exists on disk at generation time
- Check for a font/encoding issue if any non-English characters are involved
- Fallback: generate a simpler plain-text/HTML report instead of blocking on ReportLab specifics

Evidence bounding boxes are misaligned

- Confirm which image size/resolution the coordinates were computed against
- Confirm the image isn't being resized between OCR and the drawing step without rescaling the coordinates too
- Draw the box on the exact same image object OCR ran on, as a sanity check, before drawing on any resized copy

Part 40 — Copy-Paste Prompt Library (100+ Prompts)

Organized by category. Every prompt below follows the GOOD example format from this handbook's writing style: specific stack, specific symptom/goal, and an explicit ask for a test. Prompts already given in full detail in Parts 16–18 are not repeated here — this library adds further prompts across every category.

Project Management (6)

1	We are a 6-person team with 5 days left, roles: Tech Lead, Frontend, OCR/CV, Rule Engine, Backend/DB, Research/QA/Pitch. Given this MVP scope [PASTE Part 4 MUST HAVE list], propose a realistic hour-by-hour schedule for tomorrow that avoids any single person blocking two others at once.
2	Here's today's standup notes: [PASTE]. Identify any hidden dependency between two team members' tasks that isn't explicit yet, and suggest how to resolve the ordering.
3	We're 40% through Day 3 and behind on the rule engine. Given our Feature Kill List [PASTE Part 4], what's the single highest-leverage cut to make right now to protect the demo?
4	Write a one-paragraph status update I can post to our team channel summarizing what's done, what's blocked, and what's next, based on this raw list of updates: [PASTE].
5	Draft a KNOWN_ISSUES.md entry for this bug: [DESCRIBE], including severity, who's affected, and a suggested workaround for the demo if it's not fixed in time.
6	Given our 5-day plan, what's the earliest point at which we should do a full end-to-end dry run of the entire demo, and why?

Architecture (6)

7	Given this pipeline [PASTE Part 5 diagram], where are the 2-3 highest-risk integration points where one member's output format might not match another's expected input? Suggest a shared schema for each.
8	Review this API contract draft [PASTE] for consistency: are field names, types, and required/optional status consistent across every endpoint that touches the same data?
9	We're deciding whether a specific feature needs its own microservice or can just be a function in the existing FastAPI app: [DESCRIBE FEATURE]. Given our 5-day timeline, what's the lower-risk choice and why?
10	Sketch a simple ASCII architecture diagram of our system based on this description: [PASTE STACK + FLOW], suitable for including in our documentation.
11	We want to add [FEATURE]. Does this fit inside our existing pipeline stages, or does it require a new stage? Explain in one paragraph.
12	Review this folder structure [PASTE] against standard FastAPI + Next.js project conventions and flag anything that will confuse a new contributor.

Frontend (6)

13	Build a reusable Tailwind + React status banner component that accepts a `status` prop of 'compliant' 'non_compliant' 'manual_review' and renders with the correct color and icon for each.
14	This page re-renders too often and feels laggy: [PASTE COMPONENT]. Identify unnecessary re-renders and fix them using memoization where appropriate, explaining each change.
15	Add a loading skeleton (not just a spinner) to this component while data is fetching: [PASTE COMPONENT].
16	This form doesn't show a validation error when the file type is wrong: [PASTE COMPONENT]. Add clear inline validation messaging.
17	Make this component keyboard-accessible (tab order, focus states) without changing its visual design: [PASTE COMPONENT].
18	Convert this desktop-only layout to be usable on a narrow mobile screen using Tailwind's responsive classes: [PASTE COMPONENT].

Backend (6)

19	Add request validation to this FastAPI endpoint using Pydantic so invalid input returns a clear 422 error instead of crashing: [PASTE ENDPOINT].
20	This endpoint is slow: [PASTE CODE]. Profile where the time is likely going given the operations involved, and suggest the smallest change to speed it up.
21	Add structured logging to this FastAPI app so I can see each request's path, status code, and processing time in the terminal.
22	Write a health-check endpoint that also verifies the database connection and OCR engine are both reachable, returning a clear status for each.
23	This endpoint sometimes returns a stale result after an update: [PASTE CODE]. Identify whether this is a caching issue and fix it.
24	Add graceful shutdown handling to this FastAPI app so in-progress requests aren't dropped abruptly during a restart.

OCR (6)

25	Compare the raw OCR output on these two versions of the same image (original vs. after my preprocessing) [PASTE BOTH RESULTS] and tell me if the preprocessing actually helped.
26	Write a function that merges OCR results from two different engines (PaddleOCR and Tesseract) on the same image, preferring the higher-confidence result per text region.
27	My OCR confidence scores all cluster suspiciously close to 1.0 even on images I know are

	bad. What could cause this, and how do I verify it?
28	Write a script that batch-runs OCR over an entire folder of test images and saves the results to a single JSON file for later comparison.
29	Given this OCR bounding box format [PASTE SAMPLE], write a function to crop just that region out of the original image as its own file, for closer inspection.
30	My OCR works on landscape photos but not portrait ones from the same phone. What's the likely cause, and how do I fix orientation handling?

Computer Vision (6)

31	Write an OpenCV function that automatically detects and crops just the label region from a photo that includes background clutter around the package.
32	Implement a simple perspective-correction function using OpenCV for a label photographed at a slight angle, given 4 approximate corner points.
33	Write a function to detect whether an image likely contains glare (a large very-bright saturated region) so we can warn the user before OCR runs.
34	Draw a semi-transparent colored overlay (not just an outline) on a specific region of an image using OpenCV, for highlighting a finding.
35	Write a function that estimates whether a photo was taken of a curved surface (e.g. a bottle) based on simple heuristics, so we can flag lower confidence automatically.
36	Resize an image to a max dimension while preserving aspect ratio, and return the scale factor so I can rescale bounding box coordinates afterward.

Rule Engine (6)

37	Given this JSON rule schema [PASTE Part 10 schema], write a loader function that validates every rule file in the /rules folder has all required keys before the app starts, failing loudly if not.
38	Write a function that generates a plain-language explanation string from a rule result object, suitable for showing directly to a non-technical inspector.
39	I need to add a new rule for [DECLARATION TYPE]. Given the existing schema and these 2 example rules [PASTE], draft the new rule JSON in the same style.
40	Write unit tests for the confidence-floor logic specifically: a field just above, just below, and exactly at the floor.
41	Review this set of rules [PASTE] for any two rules that could produce contradictory results on the same field, and suggest how to resolve the conflict.
42	Add a `rule_version` field to our schema and a loader check that logs which version of each rule was active for a given inspection, for auditability.

Database (6)

43	Write a migration script that adds a new column to the existing `inspections` table without losing existing data.
44	Write a function to export all inspection records to a CSV file for the team's QA spreadsheet.
45	My SQLite database is locked and writes are failing intermittently. What's the likely cause given FastAPI's async request handling, and how do I fix it?
46	Write a simple backup function that copies the SQLite database file to a timestamped backup before any risky manual change.
47	Write a query to fetch the 10 most recent inspections with a `manual_review_required` status, most recent first.
48	Add a basic index to the `inspections` table on `created_at` and explain when this would actually matter at our scale (be honest if it doesn't yet).

Testing (6)

49	Write a pytest fixture that provides a temporary SQLite database for each test so tests don't interfere with each other.
50	Given this function [PASTE], identify the edge cases I haven't tested yet and write tests for each.
51	Write an integration test that uploads a real sample image through the full /inspect endpoint and asserts the response has the expected shape.
52	Review this test suite [PASTE] and tell me honestly whether it actually tests behavior or just re-asserts the implementation.
53	Write a test that deliberately feeds the rule engine a field with a confidence score of exactly 0 and confirms it degrades to UNCERTAIN, not a crash.
54	Generate 5 additional adversarial test image descriptions (not code — just descriptions) we should manually create, beyond the 22 in Part 22, specific to Indian retail packaging.

Git (6)

55	I have uncommitted changes in 6 files but only want to commit 2 of them right now. Show me exactly how to stage just those 2.
56	Explain, in plain language, the difference between `git merge` and `git rebase` and which one is safer for beginners on a shared team branch.
57	I need to see exactly what changed in the last 3 commits on this branch. Give me the exact command.
58	Write a .gitignore file appropriate for a Next.js + FastAPI + Python project, making

	sure .env and node_modules are excluded.
59	I accidentally committed a large binary test image to the repo. How do I remove it going forward without breaking the team's history too badly, given our timeline?
60	Give me the exact command to see which teammate last modified a specific line in a file, so I know who to ask about it.

Security (6)

61	Scan this FastAPI file upload endpoint [PASTE] for path traversal vulnerabilities specifically and show me the fix.
62	Is storing an OCR API key directly in a Next.js frontend environment variable safe, or does it leak to the browser? Explain and fix if needed.
63	Add basic rate limiting to this endpoint appropriate for a hackathon demo (not production-grade) to prevent accidental abuse during judging.
64	Review this SQL query [PASTE] for injection risk and rewrite it safely using parameterized queries.
65	What's the minimum viable CORS configuration for our demo that's safer than <code>allow_origins=['*']</code> but won't cost us debugging time?
66	Write a checklist I can run through in 10 minutes on Day 5 to catch the most common beginner security mistakes in a FastAPI + Next.js app.

Documentation (6)

67	Write a CONTRIBUTING.md section explaining our branch naming and commit message convention from Part 21, in plain language for teammates new to Git.
68	Summarize this rule engine module [PASTE CODE] into a short docstring explaining what it does, its inputs, and its outputs.
69	Write an ARCHITECTURE.md section explaining why the rule engine is separate from the AI/OCR layer, for someone who's never seen the codebase.
70	Turn these raw meeting notes [PASTE] into a clean, dated changelog entry.
71	Write a short 'known limitations' section for our README based on Part 36, in our own words.
72	Generate a table of contents for our project's README given these section headers: [PASTE].

Research (6)

73	Search for the current official Department of Consumer Affairs page hosting the Legal Metrology (Packaged Commodities) Rules, 2011 and confirm the most recent amendment
----	--

	date.
74	Search for any existing commercial or government tool that specifically checks Legal Metrology compliance from a photo, in India. Report what you actually find, and say clearly if you find nothing direct.
75	Search for the current pip install command for the OCR library we're using, since library APIs can change between versions.
76	Search for whether our chosen coding agent/LLM tool's pricing or free-tier limits have changed since we last checked.
77	Search for any recent (last 6 months) amendment to the Legal Metrology (Packaged Commodities) Rules, 2011 that we might have missed.
78	Search for how other government departments in India have deployed AI-assisted inspection tools, for roadmap inspiration — report real examples only.

Pitch (6)

79	Tighten this pitch paragraph [PASTE] to be 30% shorter without losing the core claim, and flag if any claim in it isn't something we've actually built or measured.
80	Given our 3-minute pitch script [PASTE Part 32], suggest where a judge's attention is most likely to drift, and how to re-hook it.
81	Write 3 alternative one-line taglines in the same spirit as 'Scan the label. Find the violation. Show the evidence.' for us to choose from.
82	Review this pitch deck slide text [PASTE] for any unsupported claim or invented statistic and flag it explicitly.
83	Suggest a natural transition line between our 'Problem' and 'Solution' pitch sections that doesn't sound like a corporate template.
84	Given these judge Q&A answers [PASTE Part 34 excerpt], which ones sound rehearsed/robotic, and how would you phrase them more naturally?

Business (6)

85	Review our Business Model Canvas [PASTE Part 28] for any box that's vague enough to invite a follow-up question we're not prepared for.
86	Explain, in plain language, the practical difference between an on-premise deployment and a SaaS deployment for a government department buyer.
87	Given our customer segments [PASTE], which one has the shortest realistic path to a first pilot, and why?
88	Draft 3 honest, non-fabricated bullet points about potential cost savings from faster inspection throughput, clearly labeled as an estimate, not a measured result.

89	What questions would a government procurement officer realistically ask that a hackathon judge might not? Help us prepare for those too.
90	Review this pricing/licensing pros-and-cons table [PASTE Part 28] for any option that's unrealistic given a 5-day-old prototype.

UI (6)

91	Suggest a color palette for our severity chips (Major/Minor) that's colorblind-accessible, not just red/amber by hue alone.
92	Review this screen [PASTE/DESCRIBE] against our personas in Part 3 — would Inspector Meena, who isn't a software person, understand this in 3 seconds?
93	Suggest a simple empty-state illustration/message for our Inspection History screen when no inspections exist yet.
94	Our Evidence View screen feels cluttered with 4+ findings. Suggest a layout that scales better without hiding information.
95	Write copy (button labels, headings, helper text) for our Image Quality screen that a non-technical user would find reassuring, not alarming, on a RETAKE result.
96	Suggest a loading-state message sequence (3-4 short phrases) to show during OCR processing so the wait feels informative, not just a spinner.

Deployment (6)

97	Given our stack (Next.js + FastAPI + SQLite + local OCR), what's the simplest way to make this accessible via a URL as a backup to a local laptop demo, and what are the exact steps?
98	Write a startup script that launches both frontend and backend with one command for demo day, so no one has to remember two separate terminal commands under pressure.
99	What environment variables would break if we moved this from a teammate's laptop to a different laptop, and how do we make that handoff painless?
100	Write a pre-demo checklist script that verifies the backend, database, and OCR engine are all reachable before we walk on stage.
101	If our deployed backup URL and our local laptop demo both fail, what's the absolute minimum needed to show the offline fallback video from Part 38 smoothly?
102	Explain the tradeoff between deploying before Day 5 (more testing time) versus deploying on Day 5 (freshest code) for a team our size, and recommend one.

Part 41 — LLM Context Files

Six small markdown files, kept up to date, that get pasted or referenced into every LLM session. This is the single biggest lever for making LLM-assisted coding reliable across a 6-person team — it turns “the LLM forgot our architecture” into a non-issue.

File	Purpose
PROJECT_CONTEXT.md	The master prompt content from Part 16, kept current — what the project is, the stack, the team, the non-negotiable rules.
ARCHITECTURE.md	The Part 5/12 pipeline diagram and the AI-vs-rule-engine boundary, plus current folder structure.
RULES.md	A human-readable summary of every implemented rule, cross-referenced to Part 9/10 — so any LLM session can see what's already covered.
API_CONTRACT.md	Every endpoint, its request/response shape, owned by Member 1 and updated the moment an endpoint changes.
KNOWN_ISSUES.md	A running list of unresolved bugs/limitations, updated at every evening review (Part 20) — stops the same bug being 're-discovered' by three different people.
TODO.md	The current day's remaining tasks per member, so a fresh LLM session (or a teammate) can see what's actually left.

Template: PROJECT_CONTEXT.md

```
# PROJECT_CONTEXT.md
## What this is
LabelGuard – SIH26034 prototype. [1-paragraph summary from Part 1.]

## Stack
Next.js/React/Tailwind (frontend), FastAPI/Python (backend),
SQLite, PaddleOCR, OpenCV, ReportLab.

## Non-negotiable rules
- The rule engine, not any LLM, makes the compliance decision.
- A missing field is UNCERTAIN, never FAIL.
- Never claim legal certification – see Part 9 language rules.

## Team
6 members, roles as in Part 18. Last updated: [date].
```

Template: API_CONTRACT.md (excerpt)

```
## POST /inspect
Request: multipart/form-data, field "file" = image
```

Response 200:

```
{
  "id": 12,
  "overall_status": "potential_non_compliance",
  "extracted_fields": { "net_quantity": {...}, ... },
  "findings": [ { "rule_id": "...", "status": "...",
                  "severity": "...", "confidence": 0.94,
                  "bounding_box": [...] } ]
}
```

Response 422: invalid file type/size

Last updated by: Member 1, [date]

NOTE

Why this matters: an LLM given only a snippet of code will confidently invent a plausible-sounding but wrong function signature or field name. An LLM given these six files as context reuses the real ones — this alone eliminates a large share of integration bugs in Part 17.

Part 42 — Claude / Coding Agent Workflow

A precise, repeatable loop for using a coding agent safely — never let it rewrite the whole repository blindly.

37. Understand the repository — let the agent look at the actual current folder structure and files, don't just describe it from memory.
38. Read the context files (Part 41) — paste or point the agent at PROJECT_CONTEXT.md, ARCHITECTURE.md, and API_CONTRACT.md before asking for a change.
39. Plan — ask the agent to describe its plan in plain language BEFORE writing code, and sanity-check the plan yourself.
40. Modify one subsystem — scope each request to one function/file/feature at a time, not “improve the whole app.”
41. Run tests — immediately, on the actual change, not just “looks right.”
42. Review the diff — read what actually changed, line by line, before accepting — not just whether it runs.
43. Commit — with a clear message (Part 21 convention).
44. Update context — if the change affects the API contract, architecture, or known issues, update those files in the same sitting.
45. Move to the next task — with fresh, accurate context for the next agent session.

NOTE — The one rule that prevents the worst outcomes

Never approve a coding-agent response that rewrites or touches files far outside the scope of what you asked. If a request to “fix this one function” comes back having also “refactored” three unrelated files, reject it and ask again more narrowly. A hackathon team cannot afford to debug an unreviewed full-repo rewrite the night before demo.

Part 43 — Code Review Checklist

Run through this before accepting ANY LLM-generated code, even your own agent's:

- Does it run? (Actually run it — don't take “this should work” on faith.)
- Is it understandable? (Could a teammate explain it back in one sentence?)
- Does it introduce unnecessary dependencies? (A new library for something 5 lines of code could do is a red flag.)
- Does it expose API keys or secrets? (Check for hardcoded strings that look like keys/tokens.)
- Does it break existing features? (Re-run the relevant tests from Part 22, not just the new one.)
- Does it have error handling? (What happens on bad input, a network failure, a missing file?)
- Is input validated? (Especially anything touching file uploads or database queries.)
- Is it tested? (At least the core logic, per Part 22.)
- Is it consistent with the architecture? (Does it respect the AI-vs-rule-engine boundary from Part 12, and match API_CONTRACT.md?)

Part 44 — Beginner Glossary

Term	Plain-language meaning
Frontend	The part of the app the user actually sees and clicks — runs in the browser.
Backend	The part of the app that runs on a server, handling logic and data the frontend can't do alone.
API	A defined way for two programs to talk to each other — like a menu of requests one program can make of another.
REST	A common style of API where you use standard web requests (GET, POST, etc.) to work with resources like 'an inspection.'
JSON	A simple, structured text format for data — like a nested list of labeled values — used to pass data between frontend and backend.
HTTP	The protocol (set of rules) browsers and servers use to send requests and responses over the internet.
GET	An HTTP request type meaning “give me data” — doesn't change anything on the server.
POST	An HTTP request type meaning “here's data, do something with it” — e.g. upload a file, create a record.
CORS	A browser security rule that blocks a webpage from calling a different server unless that server explicitly allows it.
OCR	Optical Character Recognition — software that reads text out of an image.
CV	Computer Vision — the broader field of software that interprets images (not just text).
LLM	Large Language Model — an AI model trained to understand and generate text (e.g. Claude, ChatGPT, Gemini).
Prompt	The text instruction you give an LLM.
Inference	The act of an AI model actually producing an output for a given input (as opposed to training it).
Model	The trained AI system itself (an OCR model, an LLM, etc.).
Embedding	A numeric representation of text/data that captures its meaning, used for similarity search — not used in this MVP's core path.
Database	Organized, reliably-stored data your app can query — like a very robust spreadsheet.
SQL	The language used to ask a database questions or make changes (e.g. “SELECT * FROM inspections”).
SQLite	A lightweight database that lives in a single file — no separate server needed.
Git	A tool that tracks every change to your code over time, so you can undo mistakes and work as a team without overwriting each other.

Term	Plain-language meaning
Branch	A separate, parallel copy of the code you can edit without affecting the main version until you're ready to merge.
Commit	A saved snapshot of your code changes, with a message describing what changed.
Merge	Combining changes from one branch into another.
Environment variable	A setting (like an API key) stored outside your code, so secrets aren't hardcoded and can differ per machine.
Dependency	Someone else's code your project relies on (a library/package).
Package	A bundle of reusable code you install (e.g. via pip or npm).
Runtime	The environment your code actually executes in (e.g. "Python 3.11 runtime").
Deployment	Making your app accessible somewhere other than your own laptop.
Endpoint	A specific URL path your backend responds to (e.g. /inspect).
Request	What the frontend sends to the backend asking for something.
Response	What the backend sends back.
HTTP 400	"Bad Request" — the request itself was malformed.
HTTP 401	"Unauthorized" — you need to log in / provide credentials.
HTTP 404	"Not Found" — that endpoint/resource doesn't exist.
HTTP 422	"Unprocessable Entity" — the request was understood but the data inside it failed validation.
HTTP 500	"Internal Server Error" — something broke inside the backend itself.

Part 45 — Exact Command Cheatsheet

A consolidated quick-reference. Full explanations for install steps are in Part 13; Git detail is in Part 21.

Task	Windows (PowerShell)	macOS / Linux
Check Git	<code>`git --version`</code>	<code>`git --version`</code>
Clone repo	<code>`git clone <url>`</code>	<code>`git clone <url>`</code>
New branch	<code>`git checkout -b <name>`</code>	<code>`git checkout -b <name>`</code>
Check Node/npm	<code>`node -v` / `npm -v`</code>	<code>`node -v` / `npm -v`</code>
Check Python	<code>`python --version`</code>	<code>`python3 --version`</code>
Create virtual env	<code>`python -m venv venv`</code>	<code>`python3 -m venv venv`</code>
Activate virtual env	<code>`.venv\Scripts\Activate.ps1`</code>	<code>`source venv/bin/activate`</code>
Install Python deps	<code>`pip install -r requirements.txt`</code>	<code>`pip install -r requirements.txt`</code>
Run FastAPI (dev)	<code>`uvicorn main:app --reload`</code>	<code>`uvicorn main:app --reload`</code>
Create Next.js app	<code>`npx create-next-app@latest`</code>	<code>`npx create-next-app@latest`</code>
Run Next.js (dev)	<code>`npm run dev`</code>	<code>`npm run dev`</code>
Install OCR (PaddleOCR)	<code>`pip install paddlepaddle paddleocr`</code>	<code>`pip install paddlepaddle paddleocr`</code>
Test an API endpoint	<code>`curl.exe -F "file=@sample.jpg" http://localhost:8000/inspect`</code>	<code>`curl -F "file=@sample.jpg" http://localhost:8000/inspect`</code>
Run Python tests	<code>`pytest`</code>	<code>`pytest`</code>
Build Next.js for prod	<code>`npm run build`</code>	<code>`npm run build`</code>
Open SQLite DB	<code>`sqlite3 labelguard.db`</code>	<code>`sqlite3 labelguard.db`</code>

Part 46 — Daily Checklist (Printable)

One table per day — print each morning, check off through the day.

DAY 1

Member	Task	Deadline	Definition of Done	✓
1 — Tech Lead	Repo + skeleton integration live	EOD	Frontend calls backend successfully	☐
2 — Frontend	Upload screen UI built	EOD	Thumbnail preview + disabled button logic works	☐
3 — OCR/CV	First OCR script working	EOD	Runs on 5+ real photos without crashing	☐
4 — Rule Engine	Field list + schema drafted	EOD	Every MVP field has a draft rule shape	☐
5 — Backend/DB	Schema + /scan endpoint	EOD	3 uploads create 3 distinct DB rows	☐
6 — Research/QA	Rules read + dataset started	EOD	1 compliant mock label exists	☐

DAY 2

Member	Task	Deadline	Definition of Done	✓
1 — Tech Lead	API contract + OCR wiring	EOD	OCR results visible via API	☐
2 — Frontend	Quality + processing screens	EOD	Upload→quality→processing flow works manually	☐
3 — OCR/CV	Quality gate + OCR wrapper	EOD	Correctly distinguishes 4 good vs 4 bad photos	☐
4 — Rule Engine	First 3 rules + tests	EOD	3 unit tests pass on real OCR	☐

Member	Task	Deadline	Definition of Done	✓
			output	
5 — Backend/DB	/extract endpoint	EOD	Real photo → stored OCR JSON in DB	☐
6 — Research/QA	Dataset done + 10 test cases	EOD	3 mock labels + ground truth spreadsheet exist	☐

DAY 3

Member	Task	Deadline	Definition of Done	✓
1 — Tech Lead	Rule engine wiring end-to-end	EOD	/inspect returns full result on all 3 demo images	☐
2 — Frontend	Extracted + Result screens	EOD	All 3 overall_status values render correctly	☐
3 — OCR/CV	Preprocessing + box-drawing start	EOD	Measurable accuracy improvement documented	☐
4 — Rule Engine	All 7 categories + aggregation	EOD	4 aggregation unit tests pass	☐
5 — Backend/DB	/evaluate endpoint	EOD	Full scan→extract→evaluate chain works	☐
6 — Research/QA	QA pass + ground truth refine	EOD	Rule engine output matches ground truth on demo set	☐

DAY 4

Member	Task	Deadline	Definition of Done	✓
1 — Tech Lead	Evidence + PDF integration	EOD	Full pipeline runs on all 3 demo packages	☐
2 — Frontend	Evidence + History screens	EOD	Bounding boxes render correctly on evidence view	☐
3 — OCR/CV	Evidence highlighting done	EOD	Color-coded boxes correct on all 3 demo cases	☐
4 — Rule Engine	Threshold tuning + explanations	EOD	Aggregated status matches team's manual judgment	☐
5 — Backend/DB	PDF + history endpoints	EOD	3 PDFs generate and open correctly	☐
6 — Research/QA	Full QA + pitch draft	EOD	All 22 test cases run at least once	☐

DAY 5

Member	Task	Deadline	Definition of Done	✓
1 — Tech Lead	Freeze + bug triage	12:00	No new features merged after noon	☐
2 — Frontend	States polish + rehearsal	EOD	Every screen has loading/empty/error states	☐
3 — OCR/CV	Lighting/device test	EOD	Demo works on the actual presentation device	☐
4 — Rule Engine	Regression test + freeze	Noon	No rule logic changes after this point	☐
5 — Backend/DB	Security pass + bug fixes	EOD	Part 26 checklist	☐

Member	Task	Deadline	Definition of Done	✓
			fully reviewed	
6 — Research/QA	Pitch + Q&A + backup video	EOD	Offline fallback video recorded and tested	☐

Part 47 — Definition of Done

Subsystem	Is DONE only if
OCR	It accepts an image · returns text · returns confidence · returns bounding boxes · handles failure gracefully · tested on 10+ real images.
Frontend	Upload works · loading state works · error state works · result renders correctly · no console-breaking errors on any of the 3 demo cases.
Rule Engine	Rules are explicit (JSON, not hidden in code) · test cases exist and pass · outputs are explainable (rule_id + source + confidence + bounding box present on every result).
Backend API	Every endpoint in API_CONTRACT.md is implemented · returns the documented shape · validates input · fails gracefully, never a raw unhandled 500 visible to the user.
Evidence Mapping	Every FAIL/UNCERTAIN finding has a bounding box · boxes are visibly correctly aligned on all 3 demo images · severity color-coding is consistent.
PDF Report	Generates without error on all 3 demo cases · all fields/findings are legible · no missing/None values displayed.
Database	Schema matches API_CONTRACT.md · no data loss across 5+ consecutive test inspections · backup exists before Day 5.
Overall Demo	Runs start-to-finish on all 3 demo packages · offline fallback tested and works · whole team has rehearsed it at least twice.

Part 48 — Feature Priority Matrix (MoSCoW)

Priority	Features
MUST	Image upload · image quality check · OCR · declaration extraction · structured data · rule engine · violation detection · evidence mapping · confidence score · human-review path · inspection history · report generation
SHOULD	Camera capture · multilingual OCR · dashboard analytics · richer evidence visualization
COULD	LLM-assisted extraction/normalization · semantic normalization · advanced CV (perspective correction) · offline mode · mobile wrapper
WON'T (this build)	Blockchain · custom-trained LLM · chatbot interface · IoT/sensors · facial recognition · real-time video surveillance · microservices · multiple frontends · complex multi-role auth · heavy cloud infrastructure

Part 49 — Risk Register

Risk	Prob.	Impact	Detection	Mitigation	Fallback	Owner
PaddleOCR install fails on a teammate's Windows machine	Medium	High	Install fails on Day 1	Test install on all laptops Day 1 morning	Switch that machine to Tesseract fallback (Part 11)	Member 3
OCR accuracy too low on real demo images	Medium	High	Day 2–3 testing	Apply preprocessing pipeline (Part 11)	Curate cleaner backup demo images (Part 23)	Member 3
Rule engine over-flags minor issues, demo looks broken	Medium	Medium	Day 3–4 testing	Tune confidence floors per rule	Widen thresholds before demo, document as known tuning gap	Member 4
Regex misses valid field format variants	High	Medium	Day 2 unit testing	Test regex in isolation before integration (Part 18)	Add explicit alternatives to the pattern	Member 4
Frontend-backend integration breaks (CORS/contract mismatch)	Medium	High	Daily standups	Maintain API_CONTRACT.md religiously (Part 41)	Member 1 mediates directly, same-day fix	Member 1
Bounding boxes misaligned on evidence view	Medium	Medium	Day 4 testing	Track image resize ratio explicitly through pipeline	Fall back to text-only finding (no box) rather than a wrong box	Member 3
PDF generation crashes on missing fields	Medium	Medium	Day 4 testing	Default placeholder values before PDF generation (Part 18)	Fall back to plain-text report	Member 5
SQLite file locked/corrupted near demo	Low	High	Day 5 pre-demo check	Backup DB file before any risky change	Restore from backup, or re-seed 3 demo cases fresh	Member 5
Live Wi-Fi fails during actual judging	Medium	High	N/A — always possible	Ensure core pipeline runs fully offline (Part 38)	Local laptop demo, then pre-recorded video fallback	Member 1

Risk	Prob.	Impact	Detection	Mitigation	Fallback	Owner
Team member's laptop fails on demo day	Low	High	N/A	Repo lives on GitHub, not one machine	Hot-spare laptop pre-configured (Part 38)	Member 1
Scope creep — team adds a should-have mid-build	Medium	High	Daily standups	Feature Kill List enforced by Tech Lead (Part 4)	Immediate rollback of the added feature	Member 1
Legal rule implemented incorrectly (misreads the Rules)	Low	High	Member 6's Day 1 legal read + cross-check	Cross-check every rule against the primary source, not LLM memory (Part 9)	Mark the specific rule UNCERTAIN/manual-review only, disclose in pitch	Member 6 + Member 4
Demo dataset labels are actually ambiguous/wrong	Medium	Medium	Day 2–3 QA	Independent teammate review of ground truth (Part 18)	Replace the specific ambiguous mock label	Member 6
Merge conflicts eat significant time	Medium	Low	Daily	Small, frequent commits + clear branch ownership (Part 21)	Escalate to Member 1 immediately rather than guessing	All
Team underestimates Day 1 environment setup time	High	Medium	Day 1 morning	Follow exact install commands in Part 13 verbatim	Fallback to a single 'golden' pre-configured laptop as reference	All
A teammate gets sick / unavailable for a day	Low	High	N/A	Cross-document work in context files (Part 41) so anyone can pick it up	Redistribute their Day tasks across the team per Part 19	Member 1
Confidence thresholds never get properly tuned	Medium	Medium	Day 4	Dedicate explicit Day 4 time to tuning against real dataset	Use conservative defaults and disclose as a limitation	Member 4
Pitch runs over time in front of judges	Medium	Medium	Rehearsal	Time the Part 32 script explicitly, cut ruthlessly	Have a pre-marked 60-second 'if we're almost out of time' cut	Member 6

Risk	Prob.	Impact	Detection	Mitigation	Fallback	Owner
					version	
Judge asks a question the team hasn't prepared for	High	Low	N/A	Rehearse Part 25/34 as a full team, not just Member 6	Answer honestly with the nearest prepared principle rather than guessing	All
Overpolishing UI eats time needed for the rule engine	Medium	Medium	Daily standups	Feature Kill List explicitly deprioritizes UI polish (Part 4)	Freeze UI changes earlier if rule engine is behind	Member 2
Evidence images accidentally include unrelated background/PII	Low	Medium	Day 2 dataset review	Review demo images specifically for this before use (Part 27)	Retake/crop the offending images	Member 6
API key accidentally committed to GitHub	Low	High	Pre-commit review	<code>`.gitignore`</code> for <code>`.env`</code> from Day 1 (Part 26)	Rotate the key immediately if it happens	Member 1
Team relies on a single paid LLM tool that becomes unavailable	Low	Medium	N/A	Keep a free-tier backup tool ready (Part 15)	Switch to backup tool, continue with slightly reduced velocity	All
Report generation is too slow for a live demo	Low	Low	Day 4 testing	Measure processing time explicitly (Part 24)	Pre-generate the demo reports as a fallback if truly needed	Member 5
Team disagrees on interpretation of an ambiguous rule mid-build	Medium	Low	Anytime	Default to the most cautious/UNCERTAIN interpretation (Part 9)	Escalate to Member 6 for a final call, document the reasoning	Member 4 + Member 6

Part 50 — SIH Judge Psychology

NOTE

This section reflects general hackathon judging principles commonly discussed in public SIH mentorship material — not confidential or insider judging criteria.

In a short judging slot, a judge typically needs to quickly form a view on:

What judges assess	How to structure the demo around it
Problem	State it in one sentence, tied directly to the actual PS wording (Part 2) — don't make them guess why this matters.
Novelty	Lead with the evidence-first + human-in-loop framing (Part 7/8), not “we used AI.”
Feasibility	Show it actually running, live, on real (even if mock) data — nothing sells feasibility like a working demo.
Impact	Tie back to inspector throughput and consistency — concrete, not abstract.
Technical depth	Be ready to go one level deeper than the pitch on request (Part 34) — especially the AI-vs-rule-engine boundary.
Validation	Show real, even if small, measured numbers (Part 24), explicitly labeled prototype-scale.
Scalability	Have the honest roadmap ready (Part 37), don't overclaim readiness.
Team capability	Let more than one team member speak/answer technical questions — a single spokesperson answering everything raises doubt about the rest of the team.

Part 51 — What Will Make This Project Lose

- Fake or invented accuracy numbers
- Hallucinated or oversimplified legal rules
- An architecture where an LLM makes the actual compliance decision
- An overbuilt UI with a broken or missing core pipeline underneath
- No evidence shown for a claimed violation
- No real testing behind the claims made in the pitch
- No fallback plan when (not if) something breaks live
- No real, working demo — only slides
- Excessive, unfocused features instead of a tight MVP
- An unreadable, cluttered, jargon-heavy pitch deck
- No credible business/deployment path
- Inability of team members to answer basic technical questions about their own system

Part 52 — What Will Make This Project Stand Out

46. Evidence-first compliance — every finding traceable to a real image region
47. A genuinely deterministic rule engine, not an AI black box
48. Real human-in-the-loop design, demonstrated live with an UNCERTAIN case
49. Confidence-aware output instead of false binary certainty
50. An actual test dataset with honestly measured results
51. Transparent, upfront disclosure of limitations
52. A fast, clean inspection workflow an inspector would actually want to use
53. A real, working report generation feature
54. A clean, rehearsed, reliable live demo
55. A credible, honestly-scoped roadmap from prototype to pilot

Part 53 — Final Project Blueprint

Layer	Final summary
Architecture	Image → quality gate → OCR → extraction → normalization → deterministic rule engine → result → evidence → human review → report → history (Part 5/12).
Frontend	Next.js + React + Tailwind, 12 screens (Part 6), all states implemented.
Backend	FastAPI, endpoints per API_CONTRACT.md (Part 41), orchestrating the pipeline.
OCR	PaddleOCR primary, Tesseract fallback (Part 11), with a preprocessing pipeline.
Rules	Versioned JSON rule files + Python evaluator, source-cited to Part 9 (Part 10).
Database	SQLite, single `inspections` table (Part 18 Member 5 schema).
Reports	ReportLab-generated one-page PDF per inspection (Part 18 Member 5).
Testing	22+ test cases across happy path, degraded input, and failure modes (Part 22).
Deployment	Local-first, with a simple backup URL and an offline fallback demo mode (Part 38).

Final folder structure

```

labelguard/
├── frontend/                # Next.js app (Member 2)
│   └── app/ ...            # 12 screens per Part 6
├── backend/                # FastAPI app (Member 1, 5)
│   ├── main.py
│   ├── ocr/                # OCR wrapper (Member 3)
│   ├── rules/              # Rule JSON + evaluator (Member 4, Part 10)
│   ├── db/                 # SQLite schema + queries (Member 5)
│   └── reports/            # PDF generation (Member 5)
├── tests/                  # pytest suite (Part 22)
├── data/                   # demo dataset + ground truth (Part 23, Member 6)
├── docs/
│   ├── PROJECT_CONTEXT.md
│   ├── ARCHITECTURE.md
│   ├── RULES.md
│   ├── API_CONTRACT.md
│   ├── KNOWN_ISSUES.md
│   └── TODO.md
└── README.md

```

Part 54 — Final Five-Day War Plan

NOTE — IF WE HAVE ONLY FIVE DAYS, DO THIS.

DAY 1 — Foundation: repo + skeleton + upload + first OCR script. Critical path: environment setup across all 6 laptops. Backup: one 'golden' reference laptop others can mirror.

DAY 2 — OCR + Extraction: reusable OCR function + structured field extraction + first 3 rules. Critical path: OCR reliability on real photos. Backup: Tesseract fallback engine.

DAY 3 — Compliance Engine: full rule engine + aggregation + end-to-end API wiring. Critical path: rule engine accuracy vs ground truth. Backup: widen confidence thresholds, favor manual review over false verdicts.

DAY 4 — Evidence + Report: bounding-box evidence + PDF report + full pipeline test. Critical path: evidence alignment. Backup: text-only finding display if boxes misalign.

DAY 5 — Testing + Pitch: regression test, freeze, rehearse pitch + Q&A, record offline fallback. Critical path: a clean, reliable, rehearsed demo. Backup: pre-recorded video of a full successful run.

Part 55 — Final Emergency Mode

NOTE — WE ARE 24 HOURS FROM DEMO AND THE SYSTEM IS BROKEN.

Cut in this exact order until something demoable remains:

10. Everything else (dashboard analytics, polish) — cut first
9. History (keep only the current case if needed)
8. Report (fall back to on-screen result only if PDF is broken)
7. Evidence (fall back to text-only findings if bounding boxes are broken)
6. Result (this is close to un-cuttable — it's the headline)
5. Rule engine (if truly broken, hardcode the 3 demo cases' expected results as a last resort, and be honest about it only if directly asked in a technical deep-dive, never in the main pitch)
4. Structured extraction (fall back to showing raw OCR text if extraction is broken)
3. OCR (fall back to the offline pre-recorded results from Part 38)
2. Upload (fall back to the offline demo mode entirely)
1. Never cut: having SOMETHING real to show, even if it's the Part 38 offline replay/video

Part 56 — References

Official Government Sources

- [Legal Metrology \(Packaged Commodities\) Rules, 2011 — official Department of Consumer Affairs page](#)
- [Consolidated Rules with all amendments \(official PDF\)](#)
- [Official FAQ on Legal Metrology \(Department of Consumer Affairs PDF\)](#)

Official Technical Documentation

- [Next.js Documentation / Learn](#)
- [FastAPI Documentation](#)
- [Python Documentation](#)
- [Pro Git Book](#)
- [OpenCV-Python Documentation](#)
- [PaddleOCR \(GitHub\)](#)
- [SQLite Documentation](#)
- [Tailwind CSS Documentation](#)
- [GitHub Docs](#)

Industry / Adjacent-Solution Sources (Part 29)

- [GlobalVision](#)
- [Artwork Flow](#)
- [iFactory](#)
- [SwitchOn DeepInspect](#)

NOTE

Only sources verified via live search at the time of writing are included. If a link changes or a page moves, re-verify before citing it in an official submission — government site structures do change.

Part 57 — Resource Priority

Priority	Resources
MUST READ TODAY (Day 1)	Part 9 (Legal Source of Truth) for everyone — 5 minutes each. Then your own role's Day 1 items from Part 14.
MUST READ TOMORROW	Whatever Part 14 lists for your role's Day 2 tasks — not before, don't front-load.
READ IF BLOCKED	The specific Part 17/39 troubleshooting entry matching your exact symptom — not the whole section.
OPTIONAL	Anything in Part 14 not marked MUST READ — read only if there's real spare time on Day 5.

Part 58 — Appendices

Appendix A — Master Prompt

See Part 16 for the full Master Developer Prompt — paste it at the start of every new LLM session for this project.

Appendix B — 100+ LLM Prompts

See Part 40 for the full categorized prompt library (18 categories, 100+ prompts), plus the 17 core-task prompts in Part 16 and the per-member prompts in Part 18.

Appendix C — API Contract (reference copy)

Endpoint	Request	Response (success)	Notes
POST /scan	multipart/form-data image file	{ id, image_path }	Validates type/size (Part 26)
POST /extract	{ id } or image reference	{ id, extracted_fields }	Wraps Member 3's OCR function
POST /evaluate	{ id, extracted_fields }	{ id, findings, overall_status }	Wraps Member 4's rule engine
POST /inspect	multipart/form-data image file	Full combined result (Part 41 example)	Orchestrates scan+extract+evaluate in one call
GET /history	—	[{ id, created_at, overall_status }, ...]	Most recent first
GET /inspection/{id}	—	Full inspection record	Includes officer_note if present
GET /report/{id}	—	PDF file	Generated on demand or served if cached

Appendix D — Database Schema (reference copy)

See Part 18 (Member 5, Day 1) for the full CREATE TABLE statement for the `inspections` table: id, created_at, image_path, extracted_fields (JSON text), rule_results (JSON text), overall_status, officer_note.

Appendix E — Rule Schema (reference copy)

See Part 10 for the full JSON rule schema (rule_id, field, source, description, check_type, pattern, severity_if_fail, confidence_floor, on_field_missing) and the evaluate_case() pseudocode.

Appendix F — Test Cases

See Part 22 for the full 22-case test matrix and QA spreadsheet structure.

Appendix G — Pitch Materials

See Part 31 (10-slide deck), Part 32 (3-minute word-for-word pitch), and Part 33 (90-second demo script).

Appendix H — Judge Q&A

See Part 25 (30 pressure-test attacks with weak/strong answers) and Part 34 (50+ categorized quick-reference Q&A).

Appendix I — Troubleshooting

See Part 17 (LLM Debugging Playbook, symptom-by-symptom) and Part 39 (decision trees for major subsystems).

Appendix J — Glossary

See Part 44 for the full beginner glossary.

Appendix K — Git Cheatsheet

See Part 21 (full Git workflow, branch strategy, commit convention, emergency recovery) and Part 45 (quick command reference).

Appendix L — Deployment Cheatsheet

See Part 13 (install instructions) and Part 45 (command cheatsheet) for setup; Part 38 for disaster recovery and offline demo mode.

Final Quality Self-Audit

Performed against the brief's own checklist before this handbook was finalized:

Check	Status	Where
Is SIH26034 accurately represented?	YES	Part 1, 2
Are legal claims sourced?	YES	Part 9, 56
Are official sources linked?	YES	Part 9, 56
Are current tools verified via live search?	YES	Part 11, 13, 15
Are all six members assigned work?	YES	Part 18
Does every member have five days of tasks?	YES	Part 18
Are there copy-paste LLM prompts?	YES	Parts 16, 17, 18, 40
Are there debugging prompts?	YES	Part 17, 39
Are there learning resources?	YES	Part 14, 57
Are URLs clickable?	YES	Throughout (hyperlinked)
Is the MVP feasible in five days?	YES, with disclosed scope limits	Part 4
Is there a feature kill list?	YES	Part 4
Is there an offline demo fallback?	YES	Part 38
Is the rule engine deterministic?	YES, explicitly non-LLM	Part 10, 12
Is evidence mapping explained?	YES	Part 7
Is human review included?	YES	Part 8
Are competitors researched rather than invented?	YES, none direct found, disclosed honestly	Part 29
Is the business model realistic?	YES, no invented revenue figures	Part 28
Is there a complete pitch?	YES	Parts 31–33
Is there judge Q&A?	YES, 30+50	Parts 25, 34
Is there a research-paper outline?	YES	Part 35

Check	Status	Where
Are limitations honestly disclosed?	YES	Part 36
Are test cases included?	YES, 22	Part 22
Are metrics defined without inventing numbers?	YES	Part 24
Can a beginner actually follow this handbook?	YES — every new concept explained in plain language first	Throughout